

# housePrice

August 29, 2026

## 1 Application 2 — House Price Prediction

### 1.1 1. Problem Definition

The objective of this application is to predict the historical sale price of a London property from information available about the property and its location.

This is a supervised regression problem.

Let  $X$  denote the property features and  $y$  denote the historical sale price.

For one property:

$$x_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T \in \mathbb{R}^d$$

For the complete dataset:

$$X \in \mathbb{R}^{N \times d}$$

where  $N$  is the number of property records and  $d$  is the number of input dimensions after feature selection but before categorical expansion.

The learning objective is to estimate a function

$$f(X) \rightarrow y$$

that can predict the price of an unseen property.

The assignment also requires the representation of the data to be explained before model training. Therefore, the notebook will trace the transformation from the original tabular record to the numerical feature representation supplied to the regression models.

```
[1]: # =====  
# 1.Import libraries and configure the experiment  
# =====  
  
import os  
import sys  
import time  
import json
```

```

import random
import platform
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import display

warnings.filterwarnings("ignore")

RANDOM_SEED = 42

np.random.seed(RANDOM_SEED)
random.seed(RANDOM_SEED)

print("Python version :", sys.version.split()[0])
print("Pandas version :", pd.__version__)
print("NumPy version  :", np.__version__)
print("OS              :", platform.system())
print("Random seed     :", RANDOM_SEED)

```

```

Python version : 3.14.6
Pandas version : 3.0.3
NumPy version  : 2.4.6
OS              : Windows
Random seed     : 42

```

## 1.2 2. Dataset Source

The selected dataset is London House Price Data from Kaggle.

Source:

<https://www.kaggle.com/datasets/jakewright/house-price-data>

The dataset contains historical London property transactions together with property characteristics, geographical variables, energy-related information, and price-related fields.

### 1.2.1 Question

Why is this dataset suitable for the house-price application?

### 1.2.2 Answer

The dataset directly represents London property transactions and provides both numerical and categorical property characteristics. It therefore supports the required transformation from raw tabular data into a numerical feature representation for regression.

```

[2]: # =====
# 2. Dataset Source and Experiment Configuration
# =====

import os
import sys
import time
import json
import random
import platform
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import display

# -----
# Warning configuration
# -----

warnings.filterwarnings(
    "ignore"
)

# -----
# Reproducibility
# -----

RANDOM_SEED = 42

np.random.seed(
    RANDOM_SEED
)

random.seed(
    RANDOM_SEED
)

# -----
# Dataset configuration
# -----

```

```

TARGET = "history_price"

DATA_PATH = (
    "../.. /DATA/"
    "kaggle_london_house_price_data.csv"
)

# -----
# Environment information
# -----

print(
    "Dataset          : London House Price Data"
)

print(
    "Kaggle URL        : "
    "https://www.kaggle.com/datasets/jakewright/house-price-data"
)

print(
    "Target            :",
    TARGET
)

print(
    "Random seed       :",
    RANDOM_SEED
)

print(
    "Python version    :",
    sys.version.split()[0]
)

print(
    "Pandas version     :",
    pd.__version__
)

print(
    "NumPy version      :",
    np.__version__
)

```

```
print(
    "Operating system :",
    platform.system()
)
```

```
Dataset          : London House Price Data
Kaggle URL       : https://www.kaggle.com/datasets/jakewright/house-price-data
Target          : history_price
Random seed      : 42
Python version   : 3.14.6
Pandas version   : 3.0.3
NumPy version    : 2.4.6
Operating system : Windows
```

### 1.3 3. Dataset Loading

The CSV file is loaded into a Pandas DataFrame.

If the original dataset contains more than 100,000 rows, a deterministic sample is created using the fixed random seed. This provides a reproducible experimental dataset while keeping model training practical.

#### 1.3.1 Question

How is the raw CSV converted into a form that can be inspected and processed?

#### 1.3.2 Answer

The CSV file is converted into a Pandas DataFrame. At this stage, the original columns and values are preserved so that data quality and representation can be examined before any modelling transformation is applied.

```
[3]: # =====
# 3. Dataset Loading
# =====

from pathlib import Path

# -----
# Resolve dataset path
# -----

DATA_PATH = Path(
    "../.. /DATA/"
    "kaggle_london_house_price_data.csv"
)
```

```

print(
    "Dataset path:"
)

print(
    DATA_PATH.resolve()
)

# -----
# Verify dataset file
# -----

if not DATA_PATH.exists():

    raise FileNotFoundError(

        "Dataset file was not found.\n\n"
        f"Expected path:\n"
        f"{DATA_PATH.resolve()}"

    )

# -----
# Load the complete dataset
# -----

df = pd.read_csv(
    DATA_PATH
)

# -----
# Verify target
# -----

if TARGET not in df.columns:

    raise ValueError(

        f"Target column '{TARGET}' "
        "was not found in the dataset.\n\n"

        "Available columns:\n"

        +

```

```

        "\n".join(
            f"- {column}"
            for column in df.columns
        )
    )

# -----
# Dataset information
# -----

print(
    "\nDataset loaded successfully."
)

print(
    "Number of observations:",
    len(df)
)

print(
    "Number of columns:",
    df.shape[1]
)

print(
    "Dataset shape:",
    df.shape
)

```

Dataset path:

C:\Users\ADMIN\Downloads\Intelligent System\assignment  
2\DATA\kaggle\_london\_house\_price\_data.csv

Dataset loaded successfully.

Number of observations: 418201

Number of columns: 28

Dataset shape: (418201, 28)

## 1.4 4. Dataset Inspection

The first inspection establishes the structure of the raw dataset through its dimensions, sample records, data types, and descriptive statistics.

### 1.4.1 Questions

1. What does one row represent?

2. What does one column represent?
3. What are the main data types?
4. Which variables may be useful for predicting price?

### 1.4.2 Answer

Each row represents one historical property transaction. Each column describes one attribute of that transaction or property.

The dataset contains both numerical and categorical information, so a single preprocessing strategy cannot be applied to every feature.

```
[4]: # =====
# 4. Dataset Inspection
# =====

print("Dataset shape:")
print(df.shape)

print("\nFirst five observations:")
display(
    df.head()
)

print("\nColumn data types:")
display(
    df.dtypes
    .to_frame("dtype")
)

print("\nDescriptive statistics:")
display(
    df.describe(
        include="all"
    ).T
)
```

Dataset shape:  
(418201, 28)

First five observations:

|   | fullAddress                                       | postcode | country | \ |
|---|---------------------------------------------------|----------|---------|---|
| 0 | Flat 9, 35 Furnival Street, London, EC4A 1JQ      | EC4A 1JQ | England |   |
| 1 | Flat 6, 35 Furnival Street, London, EC4A 1JQ      | EC4A 1JQ | England |   |
| 2 | Flat 35, Octavia House, Medway Street, London,... | SW1P 2TA | England |   |

|   |                                                 |         |         |
|---|-------------------------------------------------|---------|---------|
| 3 | 24 Chester Court, Lomond Grove, London, SE5 7HS | SE5 7HS | England |
| 4 | 18 Alexandra Gardens, London, N10 3RL           | N10 3RL | England |

|   | outcode | latitude  | longitude | bathrooms | bedrooms | floorAreaSqM | \ |
|---|---------|-----------|-----------|-----------|----------|--------------|---|
| 0 | EC4A    | 51.517282 | -0.110314 | 1.0       | 1.0      | 45.0         |   |
| 1 | EC4A    | 51.517282 | -0.110314 | NaN       | NaN      | NaN          |   |
| 2 | SW1P    | 51.495505 | -0.132379 | 2.0       | 2.0      | 71.0         |   |
| 3 | SE5     | 51.478185 | -0.092201 | 1.0       | 1.0      | 64.0         |   |
| 4 | N10     | 51.588774 | -0.139599 | 1.0       | 4.0      | 137.0        |   |

|   | livingRooms | ... | saleEstimate_upperPrice | saleEstimate_confidenceLevel | \ |
|---|-------------|-----|-------------------------|------------------------------|---|
| 0 | 1.0         | ... | 630000.0                | HIGH                         |   |
| 1 | NaN         | ... | 660000.0                | MEDIUM                       |   |
| 2 | 1.0         | ... | 834000.0                | MEDIUM                       |   |
| 3 | 1.0         | ... | 407000.0                | HIGH                         |   |
| 4 | 2.0         | ... | 1324000.0               | HIGH                         |   |

|   | saleEstimate_ingestedAt  | saleEstimate_valueChange.numericChange | \ |
|---|--------------------------|----------------------------------------|---|
| 0 | 2024-10-07T13:26:59.894Z | 244000.0                               |   |
| 1 | 2024-10-07T13:26:59.894Z | 425000.0                               |   |
| 2 | 2025-01-10T11:04:57.114Z | 49000.0                                |   |
| 3 | 2024-10-07T13:26:59.894Z | 28000.0                                |   |
| 4 | 2024-10-07T13:26:59.894Z | 81000.0                                |   |

|   | saleEstimate_valueChange.percentageChange | \ |
|---|-------------------------------------------|---|
| 0 | 68.539326                                 |   |
| 1 | 242.857143                                |   |
| 2 | 6.901408                                  |   |
| 3 | 7.777778                                  |   |
| 4 | 6.864407                                  |   |

|   | saleEstimate_valueChange.saleDate | history_date | history_price | \ |
|---|-----------------------------------|--------------|---------------|---|
| 0 | 2010-03-30                        | 1995-01-02   | 830000        |   |
| 1 | 2000-05-26                        | 1995-01-02   | 830000        |   |
| 2 | 2019-09-04                        | 1995-01-03   | 249950        |   |
| 3 | 2024-01-25                        | 1995-01-03   | 32000         |   |
| 4 | 2022-12-14                        | 1995-01-03   | 133000        |   |

|   | history_percentageChange | history_numericChange |
|---|--------------------------|-----------------------|
| 0 | NaN                      | NaN                   |
| 1 | NaN                      | NaN                   |
| 2 | NaN                      | NaN                   |
| 3 | NaN                      | NaN                   |
| 4 | NaN                      | NaN                   |

[5 rows x 28 columns]

Column data types:

|                                           | dtype   |
|-------------------------------------------|---------|
| fullAddress                               | str     |
| postcode                                  | str     |
| country                                   | str     |
| outcode                                   | str     |
| latitude                                  | float64 |
| longitude                                 | float64 |
| bathrooms                                 | float64 |
| bedrooms                                  | float64 |
| floorAreaSqM                              | float64 |
| livingRooms                               | float64 |
| tenure                                    | str     |
| propertyType                              | str     |
| currentEnergyRating                       | str     |
| rentEstimate_lowerPrice                   | float64 |
| rentEstimate_currentPrice                 | float64 |
| rentEstimate_upperPrice                   | float64 |
| saleEstimate_lowerPrice                   | float64 |
| saleEstimate_currentPrice                 | float64 |
| saleEstimate_upperPrice                   | float64 |
| saleEstimate_confidenceLevel              | str     |
| saleEstimate_ingestedAt                   | str     |
| saleEstimate_valueChange.numericChange    | float64 |
| saleEstimate_valueChange.percentageChange | float64 |
| saleEstimate_valueChange.saleDate         | str     |
| history_date                              | str     |
| history_price                             | int64   |
| history_percentageChange                  | float64 |
| history_numericChange                     | float64 |

Descriptive statistics:

|                     | count    | unique | \ |
|---------------------|----------|--------|---|
| fullAddress         | 418201   | 137760 |   |
| postcode            | 418201   | 48392  |   |
| country             | 418201   | 1      |   |
| outcode             | 418201   | 168    |   |
| latitude            | 418201.0 | NaN    |   |
| longitude           | 418201.0 | NaN    |   |
| bathrooms           | 340270.0 | NaN    |   |
| bedrooms            | 377665.0 | NaN    |   |
| floorAreaSqM        | 392991.0 | NaN    |   |
| livingRooms         | 357701.0 | NaN    |   |
| tenure              | 406689   | 4      |   |
| propertyType        | 417065   | 19     |   |
| currentEnergyRating | 333675   | 7      |   |

|                                           |          |      |
|-------------------------------------------|----------|------|
| rentEstimate_lowerPrice                   | 416460.0 | NaN  |
| rentEstimate_currentPrice                 | 416460.0 | NaN  |
| rentEstimate_upperPrice                   | 416460.0 | NaN  |
| saleEstimate_lowerPrice                   | 417561.0 | NaN  |
| saleEstimate_currentPrice                 | 417561.0 | NaN  |
| saleEstimate_upperPrice                   | 417561.0 | NaN  |
| saleEstimate_confidenceLevel              | 417561   | 3    |
| saleEstimate_ingestedAt                   | 417561   | 4    |
| saleEstimate_valueChange.numericChange    | 417561.0 | NaN  |
| saleEstimate_valueChange.percentageChange | 417561.0 | NaN  |
| saleEstimate_valueChange.saleDate         | 417561   | 6241 |
| history_date                              | 418201   | 8325 |
| history_price                             | 418201.0 | NaN  |
| history_percentageChange                  | 237982.0 | NaN  |
| history_numericChange                     | 237982.0 | NaN  |

|                         |                                 |
|-------------------------|---------------------------------|
| ↪ top \                 |                                 |
| fullAddress             | 74 Western Beach Apartments, 36 |
| ↪ Hanover Avenue...     |                                 |
| postcode                |                                 |
| ↪ SW1P 3AN              |                                 |
| country                 |                                 |
| ↪ England               |                                 |
| outcode                 |                                 |
| ↪ SE18                  |                                 |
| latitude                |                                 |
| ↪ NaN                   |                                 |
| longitude               |                                 |
| ↪ NaN                   |                                 |
| bathrooms               |                                 |
| ↪ NaN                   |                                 |
| bedrooms                |                                 |
| ↪ NaN                   |                                 |
| floorAreaSqM            |                                 |
| ↪ NaN                   |                                 |
| livingRooms             |                                 |
| ↪ NaN                   |                                 |
| tenure                  |                                 |
| ↪ Leasehold             |                                 |
| propertyType            |                                 |
| ↪ Purpose Built Flat    |                                 |
| currentEnergyRating     |                                 |
| ↪ D                     |                                 |
| rentEstimate_lowerPrice |                                 |
| ↪ NaN                   |                                 |

|                                           |                            |   |
|-------------------------------------------|----------------------------|---|
| rentEstimate_currentPrice                 | ↳ NaN                      | ⌋ |
| rentEstimate_upperPrice                   | ↳ NaN                      | ⌋ |
| saleEstimate_lowerPrice                   | ↳ NaN                      | ⌋ |
| saleEstimate_currentPrice                 | ↳ NaN                      | ⌋ |
| saleEstimate_upperPrice                   | ↳ NaN                      | ⌋ |
| saleEstimate_confidenceLevel              | ↳ HIGH                     | ⌋ |
| saleEstimate_ingestedAt                   | ↳ 2024-10-07T13:26:59.894Z | ⌋ |
| saleEstimate_valueChange.numericChange    | ↳ NaN                      | ⌋ |
| saleEstimate_valueChange.percentageChange | ↳ NaN                      | ⌋ |
| saleEstimate_valueChange.saleDate         | ↳ 2021-06-30               | ⌋ |
| history_date                              | ↳ 2021-06-30               | ⌋ |
| history_price                             | ↳ NaN                      | ⌋ |
| history_percentageChange                  | ↳ NaN                      | ⌋ |
| history_numericChange                     | ↳ NaN                      | ⌋ |

|                           | freq   | mean \      |
|---------------------------|--------|-------------|
| fullAddress               | 224    | NaN         |
| postcode                  | 366    | NaN         |
| country                   | 418201 | NaN         |
| outcode                   | 8307   | NaN         |
| latitude                  | NaN    | 51.507522   |
| longitude                 | NaN    | -0.101426   |
| bathrooms                 | NaN    | 1.444265    |
| bedrooms                  | NaN    | 2.474884    |
| floorAreaSqM              | NaN    | 98.926543   |
| livingRooms               | NaN    | 1.282275    |
| tenure                    | 247137 | NaN         |
| propertyType              | 110096 | NaN         |
| currentEnergyRating       | 135656 | NaN         |
| rentEstimate_lowerPrice   | NaN    | 3299.054171 |
| rentEstimate_currentPrice | NaN    | 3639.487466 |
| rentEstimate_upperPrice   | NaN    | 3980.117298 |

|                                           |        |               |
|-------------------------------------------|--------|---------------|
| saleEstimate_lowerPrice                   | NaN    | 831269.215755 |
| saleEstimate_currentPrice                 | NaN    | 904518.762528 |
| saleEstimate_upperPrice                   | NaN    | 977763.270516 |
| saleEstimate_confidenceLevel              | 300772 | NaN           |
| saleEstimate_ingestedAt                   | 282304 | NaN           |
| saleEstimate_valueChange.numericChange    | NaN    | 28860.767648  |
| saleEstimate_valueChange.percentageChange | NaN    | 29.278605     |
| saleEstimate_valueChange.saleDate         | 2170   | NaN           |
| history_date                              | 1071   | NaN           |
| history_price                             | NaN    | 639584.00439  |
| history_percentageChange                  | NaN    | 105.877778    |
| history_numericChange                     | NaN    | 252874.079405 |

|                                           |                |              |   |
|-------------------------------------------|----------------|--------------|---|
|                                           | std            | min          | \ |
| fullAddress                               | NaN            | NaN          |   |
| postcode                                  | NaN            | NaN          |   |
| country                                   | NaN            | NaN          |   |
| outcode                                   | NaN            | NaN          |   |
| latitude                                  | 0.056171       | 51.385708    |   |
| longitude                                 | 0.08693        | -0.347055    |   |
| bathrooms                                 | 0.724478       | 1.0          |   |
| bedrooms                                  | 1.160506       | 1.0          |   |
| floorAreaSqM                              | 55.938911      | 10.0         |   |
| livingRooms                               | 0.574412       | 1.0          |   |
| tenure                                    | NaN            | NaN          |   |
| propertyType                              | NaN            | NaN          |   |
| currentEnergyRating                       | NaN            | NaN          |   |
| rentEstimate_lowerPrice                   | 3168.625404    | 300.0        |   |
| rentEstimate_currentPrice                 | 3487.801945    | 350.0        |   |
| rentEstimate_upperPrice                   | 3808.508868    | 400.0        |   |
| saleEstimate_lowerPrice                   | 808843.379724  | 81000.0      |   |
| saleEstimate_currentPrice                 | 920291.664239  | 89000.0      |   |
| saleEstimate_upperPrice                   | 1037913.051264 | 93000.0      |   |
| saleEstimate_confidenceLevel              | NaN            | NaN          |   |
| saleEstimate_ingestedAt                   | NaN            | NaN          |   |
| saleEstimate_valueChange.numericChange    | 1080893.492314 | -105377000.0 |   |
| saleEstimate_valueChange.percentageChange | 362.350731     | -99.763196   |   |
| saleEstimate_valueChange.saleDate         | NaN            | NaN          |   |
| history_date                              | NaN            | NaN          |   |
| history_price                             | 1327933.407839 | 10000.0      |   |
| history_percentageChange                  | 338.849944     | -99.89011    |   |
| history_numericChange                     | 802403.242233  | -73900000.0  |   |

|             |     |     |     |   |
|-------------|-----|-----|-----|---|
|             | 25% | 50% | 75% | \ |
| fullAddress | NaN | NaN | NaN |   |
| postcode    | NaN | NaN | NaN |   |
| country     | NaN | NaN | NaN |   |
| outcode     | NaN | NaN | NaN |   |

|                                           |           |           |            |
|-------------------------------------------|-----------|-----------|------------|
| latitude                                  | 51.464315 | 51.501581 | 51.548515  |
| longitude                                 | -0.155996 | -0.108482 | -0.043582  |
| bathrooms                                 | 1.0       | 1.0       | 2.0        |
| bedrooms                                  | 2.0       | 2.0       | 3.0        |
| floorAreaSqM                              | 63.0      | 84.0      | 116.0      |
| livingRooms                               | 1.0       | 1.0       | 1.0        |
| tenure                                    | NaN       | NaN       | NaN        |
| propertyType                              | NaN       | NaN       | NaN        |
| currentEnergyRating                       | NaN       | NaN       | NaN        |
| rentEstimate_lowerPrice                   | 1800.0    | 2350.0    | 3550.0     |
| rentEstimate_currentPrice                 | 1950.0    | 2600.0    | 3950.0     |
| rentEstimate_upperPrice                   | 2100.0    | 2850.0    | 4300.0     |
| saleEstimate_lowerPrice                   | 416000.0  | 581000.0  | 913000.0   |
| saleEstimate_currentPrice                 | 446000.0  | 622000.0  | 984000.0   |
| saleEstimate_upperPrice                   | 475000.0  | 663000.0  | 1054000.0  |
| saleEstimate_confidenceLevel              | NaN       | NaN       | NaN        |
| saleEstimate_ingestedAt                   | NaN       | NaN       | NaN        |
| saleEstimate_valueChange.numericChange    | 11000.0   | 35000.0   | 86000.0    |
| saleEstimate_valueChange.percentageChange | 2.105263  | 6.26087   | 13.554987  |
| saleEstimate_valueChange.saleDate         | NaN       | NaN       | NaN        |
| history_date                              | NaN       | NaN       | NaN        |
| history_price                             | 228000.0  | 400000.0  | 665000.0   |
| history_percentageChange                  | 17.833333 | 52.857143 | 114.285714 |
| history_numericChange                     | 48495.0   | 130000.0  | 287000.0   |

|                                        |            |
|----------------------------------------|------------|
|                                        | max        |
| fullAddress                            | NaN        |
| postcode                               | NaN        |
| country                                | NaN        |
| outcode                                | NaN        |
| latitude                               | 51.665823  |
| longitude                              | 0.139249   |
| bathrooms                              | 9.0        |
| bedrooms                               | 9.0        |
| floorAreaSqM                           | 500.0      |
| livingRooms                            | 9.0        |
| tenure                                 | NaN        |
| propertyType                           | NaN        |
| currentEnergyRating                    | NaN        |
| rentEstimate_lowerPrice                | 111500.0   |
| rentEstimate_currentPrice              | 121200.0   |
| rentEstimate_upperPrice                | 130900.0   |
| saleEstimate_lowerPrice                | 23376000.0 |
| saleEstimate_currentPrice              | 29220000.0 |
| saleEstimate_upperPrice                | 35064000.0 |
| saleEstimate_confidenceLevel           | NaN        |
| saleEstimate_ingestedAt                | NaN        |
| saleEstimate_valueChange.numericChange | 22681000.0 |

|                                           |             |
|-------------------------------------------|-------------|
| saleEstimate_valueChange.percentageChange | 48860.0     |
| saleEstimate_valueChange.saleDate         | NaN         |
| history_date                              | NaN         |
| history_price                             | 105800000.0 |
| history_percentageChange                  | 48800.77    |
| history_numericChange                     | 63820000.0  |

## 1.5 5. Data-Quality Analysis

Data quality is examined before model development.

The analysis focuses on:

- missing values;
- duplicated records;
- invalid values;
- statistical outliers;
- target distribution.

### 1.5.1 Question

Are there data-quality problems that could make the regression results unreliable?

### 1.5.2 Answer

The notebook identifies problems first and applies cleaning only where there is a clear reason to do so. This prevents unnecessary modification of the original observations.

```
[5]: # =====
# 5. Overall data-quality summary
# =====

quality_summary = pd.DataFrame({

    "Rows":
        [len(df)],

    "Columns":
        [df.shape[1]],

    "Missing Values":
        [int(df.isna().sum().sum())],

    "Duplicate Rows":
        [int(df.duplicated().sum())]

})

display(
    quality_summary
```

```
)
```

|   | Rows   | Columns | Missing Values | Duplicate Rows |
|---|--------|---------|----------------|----------------|
| 0 | 418201 | 28      | 672132         | 7              |

## 1.6 6. Missing-Value Analysis

Missing values are analysed at the feature level.

### 1.6.1 Question

Which features contain missing information, and should those observations be removed?

### 1.6.2 Answer

Removing every row with a missing value can unnecessarily reduce the available training data.

For this reason, missing values will be handled inside the preprocessing pipeline:

- numerical features will use median imputation;
- categorical features will use most-frequent imputation.

The imputation parameters will be learned only from the training data.

```
[6]: # =====  
# 6. Missing-value analysis  
# =====  
  
missing_summary = (  
    df.isna()  
    .sum()  
    .to_frame(  
        "Missing Count"  
    )  
)  
  
missing_summary["Percentage"] = (  
    missing_summary["Missing Count"]  
    / len(df)  
    * 100  
)  
  
missing_summary = (  
    missing_summary  
    .query(  
        "`Missing Count` > 0"  
    )  
    .sort_values(  
        "Missing Count",  
        ascending=False
```

```

    )
)

if missing_summary.empty:

    print(
        "No missing values were found."
    )

else:

    display(
        missing_summary
    )

```

|                                           | Missing Count | Percentage |
|-------------------------------------------|---------------|------------|
| history_percentageChange                  | 180219        | 43.093871  |
| history_numericChange                     | 180219        | 43.093871  |
| currentEnergyRating                       | 84526         | 20.211812  |
| bathrooms                                 | 77931         | 18.634819  |
| livingRooms                               | 60500         | 14.466728  |
| bedrooms                                  | 40536         | 9.692947   |
| floorAreaSqM                              | 25210         | 6.028202   |
| tenure                                    | 11512         | 2.752743   |
| rentEstimate_upperPrice                   | 1741          | 0.416307   |
| rentEstimate_currentPrice                 | 1741          | 0.416307   |
| rentEstimate_lowerPrice                   | 1741          | 0.416307   |
| propertyType                              | 1136          | 0.271640   |
| saleEstimate_currentPrice                 | 640           | 0.153036   |
| saleEstimate_lowerPrice                   | 640           | 0.153036   |
| saleEstimate_upperPrice                   | 640           | 0.153036   |
| saleEstimate_confidenceLevel              | 640           | 0.153036   |
| saleEstimate_valueChange.numericChange    | 640           | 0.153036   |
| saleEstimate_ingestedAt                   | 640           | 0.153036   |
| saleEstimate_valueChange.saleDate         | 640           | 0.153036   |
| saleEstimate_valueChange.percentageChange | 640           | 0.153036   |

## 1.7 7. Duplicate Analysis

Exact duplicate rows are checked before splitting the data.

### 1.7.1 Question

Are there repeated records that could give some properties excessive influence during training?

### 1.7.2 Answer

Exact duplicates should normally be removed because the same observation should not appear multiple times in the learning data.

The duplicate count is measured first, and only exact duplicates are removed.

```
[7]: # =====  
# 7. Duplicate analysis and removal  
# =====  
  
duplicate_count = (  
    df.duplicated()  
    .sum()  
)  
  
print(  
    "Duplicate rows:",  
    duplicate_count  
)  
  
rows_before = len(df)  
  
df = (  
    df  
    .drop_duplicates()  
    .reset_index(  
        drop=True  
    )  
)  
  
print(  
    "Rows removed:",  
    rows_before - len(df)  
)  
  
print(  
    "Rows after duplicate removal:",  
    len(df)  
)
```

Duplicate rows: 7

Rows removed: 7

Rows after duplicate removal: 418194

## 1.8 8. Invalid-Value Analysis

Invalid values are different from statistical outliers.

An outlier may represent a real expensive or unusual property, while an invalid value violates a basic domain constraint.

### 1.8.1 Questions

Which observations are clearly invalid?

How should these observations be handled?

### 1.8.2 Answer

Examples of invalid values include non-positive property prices, non-positive floor area, and negative room counts.

These observations cannot represent valid house-price training examples and are therefore removed when the corresponding feature is available.

```
[8]: # =====  
# 8. Invalid-value analysis  
# =====  
  
invalid_checks = {}  
  
if TARGET in df.columns:  
    invalid_checks[TARGET] = (  
        df[TARGET].notna()  
        &  
        (df[TARGET] <= 0)  
    )  
  
for column in [  
    "floorAreaSqM",  
    "bedrooms",  
    "bathrooms",  
    "livingRooms"  
]:  
    if column in df.columns:  
        invalid_checks[column] = (  
            df[column].notna()  
            &  
            (df[column] < 0)  
        )  
  
invalid_results = []  
  
for column, mask in invalid_checks.items():
```

```

invalid_results.append({

    "Feature":
        column,

    "Invalid Count":
        int(mask.sum()),

    "Percentage":
        round(
            mask.mean() * 100,
            3
        )

})

invalid_df = pd.DataFrame(
    invalid_results
)

display(
    invalid_df
)

```

|   | Feature       | Invalid Count | Percentage |
|---|---------------|---------------|------------|
| 0 | history_price | 0             | 0.0        |
| 1 | floorAreaSqM  | 0             | 0.0        |
| 2 | bedrooms      | 0             | 0.0        |
| 3 | bathrooms     | 0             | 0.0        |
| 4 | livingRooms   | 0             | 0.0        |

```

[9]: # =====
# Remove clearly invalid observations
# =====

before_cleaning = len(df)

df = df[
    df[TARGET].notna()
    &
    (df[TARGET] > 0)
]

```

```

if "floorAreaSqM" in df.columns:

    df = df[
        df["floorAreaSqM"].isna()
        |
        (df["floorAreaSqM"] > 0)
    ]

for column in [
    "bedrooms",
    "bathrooms",
    "livingRooms"
]:

    if column in df.columns:

        df = df[
            df[column].isna()
            |
            (df[column] >= 0)
        ]

df = df.reset_index(
    drop=True
)

print(
    "Rows before invalid-value cleaning:",
    before_cleaning
)

print(
    "Rows after cleaning:",
    len(df)
)

print(
    "Rows removed:",
    before_cleaning - len(df)
)

```

```

Rows before invalid-value cleaning: 418194
Rows after cleaning: 418194
Rows removed: 0

```

## 1.9 9. Outlier Analysis

### 1.9.1 Question

Which numerical variables contain unusually extreme observations?

The Interquartile Range method is used to identify statistical outliers.

$$IQR = Q_3 - Q_1$$

The lower and upper bounds are:

$$L = Q_1 - 1.5(IQR)$$

$$U = Q_3 + 1.5(IQR)$$

An observation outside these limits is flagged as an outlier.

### 1.9.2 Answer

Outlier detection is used to understand the data rather than automatically delete all extreme observations. In real-estate data, an unusually expensive or large property can be a legitimate observation.

```
[10]: # =====  
# 9. IQR-based outlier analysis  
# =====  
  
outlier_features = [  
    column  
  
    for column in [  
        TARGET,  
        "floorAreaSqM",  
        "bedrooms",  
        "bathrooms",  
        "livingRooms"  
    ]  
  
    if column in df.columns  
]  
  
outlier_results = []
```

```

for column in outlier_features:

    values = (
        df[column]
        .dropna()
    )

    if values.empty:

        continue

    q1 = values.quantile(
        0.25
    )

    q3 = values.quantile(
        0.75
    )

    iqr = (
        q3 - q1
    )

    lower = (
        q1 - 1.5 * iqr
    )

    upper = (
        q3 + 1.5 * iqr
    )

    mask = (
        (values < lower)
        |
        (values > upper)
    )

    outlier_results.append({

        "Feature":
            column,

```

```

        "Outlier Count":
            int(mask.sum()),

        "Percentage":
            round(
                mask.mean() * 100,
                2
            )
    })

outlier_df = pd.DataFrame(
    outlier_results
)

display(
    outlier_df
)

```

|   | Feature       | Outlier Count | Percentage |
|---|---------------|---------------|------------|
| 0 | history_price | 34699         | 8.30       |
| 1 | floorAreaSqM  | 23355         | 5.94       |
| 2 | bedrooms      | 22333         | 5.91       |
| 3 | bathrooms     | 6085          | 1.79       |
| 4 | livingRooms   | 82907         | 23.18      |

```

[11]: # =====
# Representative outlier visualization
# =====

for column in [
    TARGET,
    "floorAreaSqM"
]:

    if column not in df.columns:

        continue

    plt.figure(
        figsize=(8, 4)
    )

    plt.boxplot(

```

```

    df[column].dropna()
)

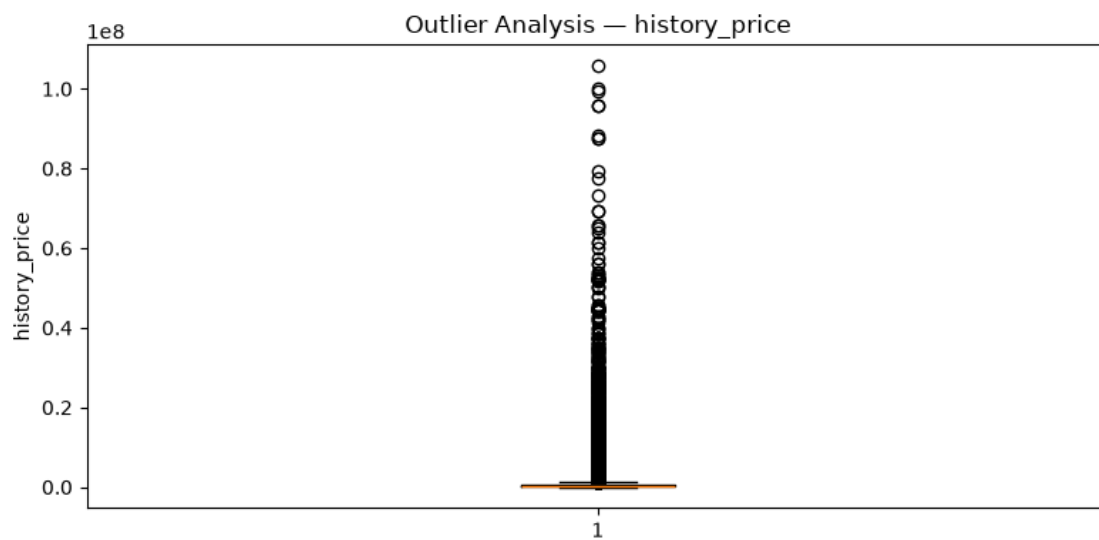
plt.title(
    f"Outlier Analysis - {column}"
)

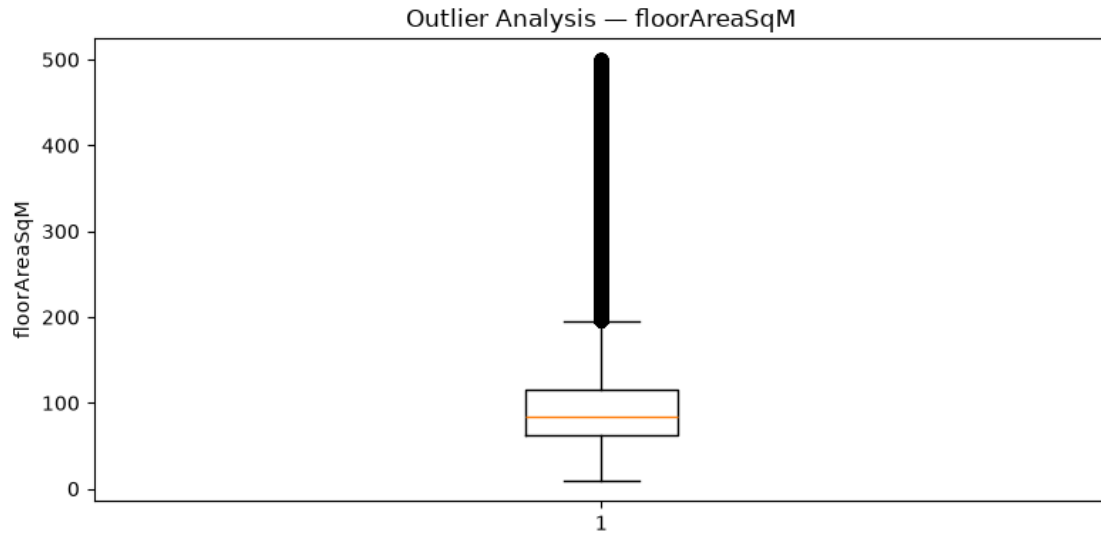
plt.ylabel(
    column
)

plt.tight_layout()

plt.show()

```





## 1.10 10. Exploratory Data Analysis

EDA is used to understand the statistical structure of the dataset before model development.

The analysis focuses on:

1. target distribution;
2. important numerical feature distributions;
3. relationships between features and price;
4. categorical effects on price;
5. numerical correlations.

### 1.10.1 Question

Which characteristics appear to contain useful information about historical house price?

### 1.10.2 Answer

The objective is not to prove causality. Instead, EDA identifies observable patterns that can motivate the choice of model family and preprocessing strategy.

### 1.10.3 Target Distribution

#### 1.10.4 Question

Is historical house price evenly distributed?

A strongly skewed target can make large prediction errors disproportionately important, which affects the interpretation of regression metrics.

```
[12]: # =====
# Target distribution
```

```

# =====

plt.figure(
    figsize=(9, 5)
)

plt.hist(
    df[TARGET].dropna(),
    bins=60
)

plt.title(
    "Distribution of Historical House Prices"
)

plt.xlabel(
    "Historical Price"
)

plt.ylabel(
    "Number of Properties"
)

plt.tight_layout()

plt.show()

display(
    df[TARGET]
    .describe()
    .to_frame(
        "history_price"
    )
)

```



```

history_price
count      4.181940e+05
mean       6.395867e+05
std        1.327944e+06
min         1.000000e+04
25%        2.280000e+05
50%        4.000000e+05
75%        6.650000e+05
max        1.058000e+08

```

[ ]:

### 1.10.5 Interpretation

The distribution should be examined for a long upper tail.

If the target is strongly right-skewed, a relatively small number of expensive properties may produce very large prediction errors.

This motivates evaluating the models with both MAE and RMSE.

### 1.10.6 Property Size and Price

#### 1.10.7 Question

How does floor area relate to historical house price?

Floor area is a direct physical characteristic of the property and is therefore an important variable for examining the relationship between property characteristics and price.

```

[13]: # =====
# Floor area versus price
# =====

if "floorAreaSqM" in df.columns:

    area_price = (
        df[
            [
                "floorAreaSqM",
                TARGET
            ]
        ].dropna()
    )

    area_price = (
        area_price
        .sample(
            min(
                20000,
                len(area_price)
            ),
            random_state=RANDOM_SEED
        )
    )

    plt.figure(
        figsize=(8, 5)
    )

    plt.scatter(

        area_price["floorAreaSqM"],

        area_price[TARGET],

        alpha=0.25

    )

    plt.title(
        "Historical Price vs Floor Area"

```

```

)

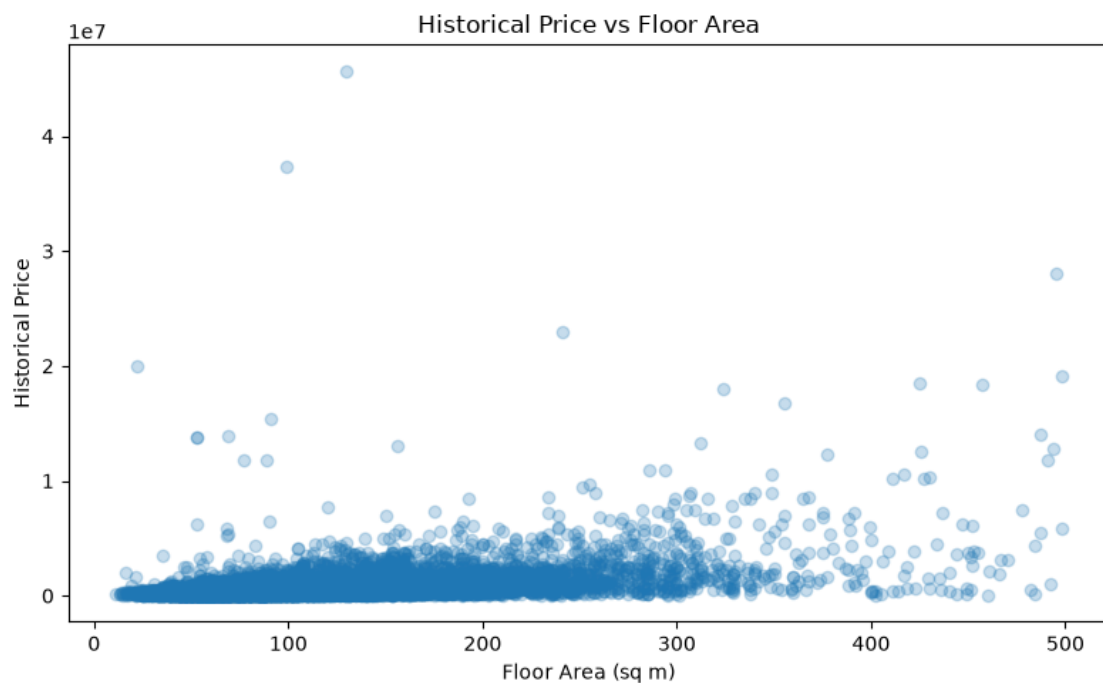
plt.xlabel(
    "Floor Area (sq m)"
)

plt.ylabel(
    "Historical Price"
)

plt.tight_layout()

plt.show()

```



### 1.10.8 Interpretation

A positive relationship between floor area and price would indicate that larger properties generally tend to have higher transaction prices.

The relationship is unlikely to be perfectly linear because property location, type, tenure, and other characteristics also contribute to price variation.

### 1.10.9 Property Type and Price

#### 1.10.10 Question

Do different property types have different typical price levels?

Property type is categorical information. Comparing its median price provides an initial indication of whether this information could be useful for prediction.

```
[14]: # =====  
# Median price by property type  
# =====  
  
if "propertyType" in df.columns:  
  
    property_type_summary = (  
  
        df[  
            [  
                "propertyType",  
                TARGET  
            ]  
        ]  
  
        .dropna()  
  
        .groupby(  
            "propertyType"  
        )[TARGET]  
  
        .median()  
  
        .sort_values(  
            ascending=False  
        )  
  
        .head(10)  
  
    )  
  
    plt.figure(  
        figsize=(10, 5)  
    )  
  
    plt.bar(  
        property_type_summary.index,  
        property_type_summary.values
```

```

)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.title(
    "Median Historical Price by Property Type"
)

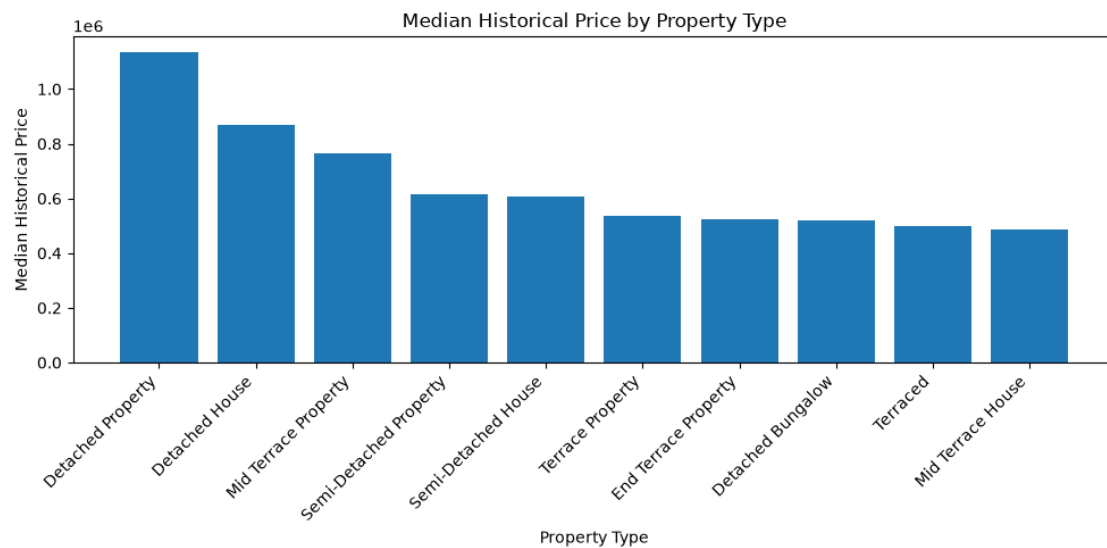
plt.xlabel(
    "Property Type"
)

plt.ylabel(
    "Median Historical Price"
)

plt.tight_layout()

plt.show()

```



### 1.10.11 Interpretation

If median prices differ substantially between property types, categorical information contributes predictive information beyond numerical measurements.

This motivates encoding categorical variables rather than discarding them.

### 1.10.12 Numerical Correlation

### 1.10.13 Question

How are the major numerical variables related to one another and to historical price?

Correlation is useful for identifying strong linear relationships and potentially redundant variables. It does not establish causality.

```
[44]: # =====  
# Correlation matrix with numerical values  
# =====  
  
correlation_candidates = [  
    TARGET,  
    "floorAreaSqM",  
    "bedrooms",  
    "bathrooms",  
    "livingRooms",  
    "latitude",  
    "longitude"  
]  
  
correlation_columns = [  
    column  
    for column in correlation_candidates  
    if column in df.columns  
]  
  
corr = (  
    df[correlation_columns]  
    .corr()  
)  
  
plt.figure(  
    figsize=(10, 8)  
)  
  
image = plt.imshow(  

```

```

    corr,
    aspect="auto"
)

# -----
# Axis labels
# -----

plt.xticks(
    range(len(corr.columns)),
    corr.columns,
    rotation=45,
    ha="right"
)

plt.yticks(
    range(len(corr.columns)),
    corr.columns
)

# -----
# Display correlation values inside each cell
# -----

for row in range(
    corr.shape[0]
):
    for col in range(
        corr.shape[1]
    ):
        value = corr.iloc[
            row,
            col
        ]

        plt.text(
            col,
            row,
            f"{value:.2f}",
            ha="center",
            va="center",
            fontsize=9
        )

```

```

# -----
# Color scale
# -----

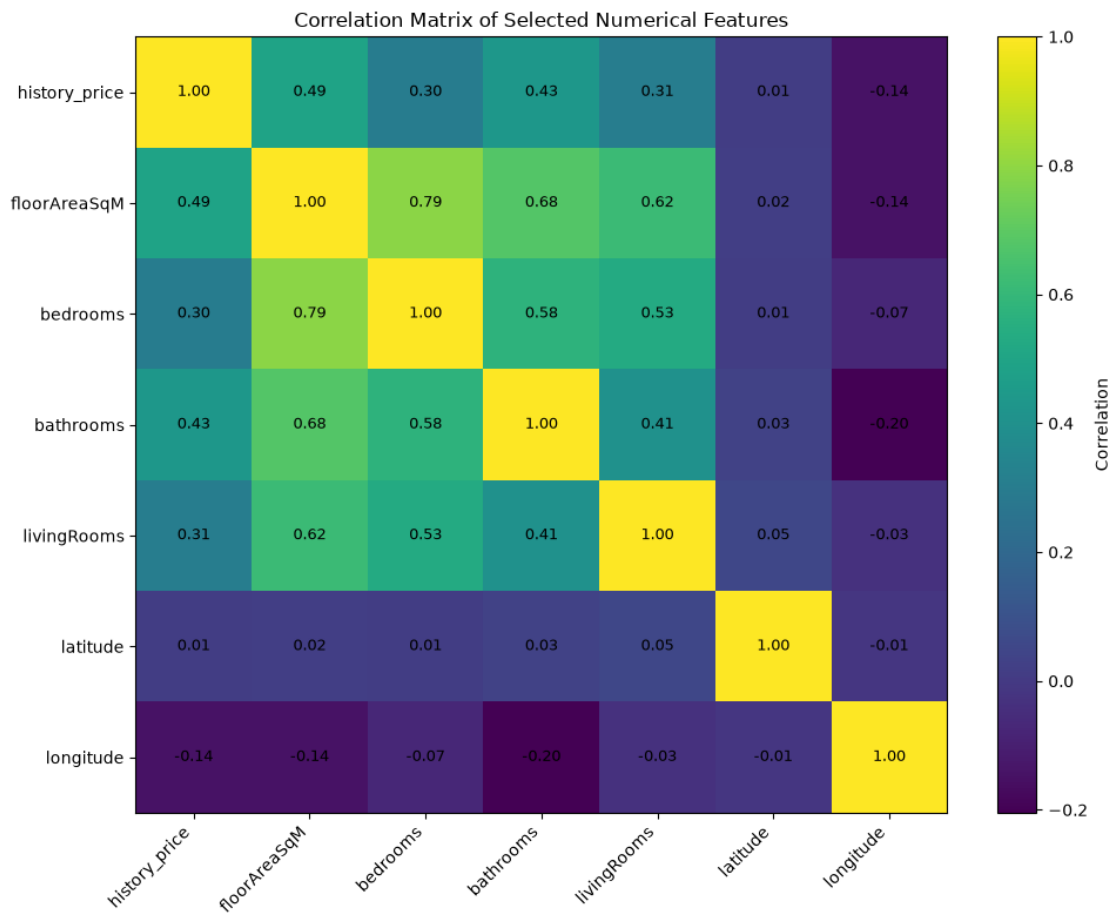
plt.colorbar(
    image,
    label="Correlation"
)

plt.title(
    "Correlation Matrix of Selected Numerical Features"
)

plt.tight_layout()

plt.show()

```



## 1.11 11. Feature Types

### 1.11.1 Question

Which features are numerical, which are categorical, and which should not be directly used as predictors?

Numerical features represent measurable quantities.

Categorical features represent discrete property characteristics and require encoding before model training.

Some columns are excluded because they are:

- the target;
- raw address identifiers;
- current price estimates;
- fields that could expose information unavailable at the historical transaction time.

### 1.11.2 Answer

The final feature set must contain variables that are available and meaningful for predicting the historical price. Excluding leakage-prone fields is therefore part of the feature-definition process, not merely a modelling choice.

```
[16]: # =====  
# 11. Define predictive features  
# =====  
  
excluded_columns = {  
    TARGET  
}  
  
# -----  
# Raw high-cardinality address fields  
# -----  
  
for column in [  
    "fullAddress",  
    "postcode"  
]:  
  
    if column in df.columns:  
  
        excluded_columns.add(  
            column  
        )
```

```

# -----
# Current market estimates
#
# These describe current estimates and may contain future
# information relative to a historical transaction.
# -----

for column in df.columns:

    lower_name = column.lower()

    if (
        "saleestimate" in lower_name
        or
        "rentestimate" in lower_name
    ):

        excluded_columns.add(
            column
        )

# -----
# Predictive matrix
# -----

candidate_features = [

    column

    for column in df.columns

    if column not in excluded_columns
]

X_raw = (
    df[
        candidate_features
    ]
    .copy()
)

```

```

y = (
    df[TARGET]
    .copy()
)

numeric_features = (

    X_raw
    .select_dtypes(
        include=np.number
    )
    .columns
    .tolist()

)

categorical_features = (

    X_raw
    .select_dtypes(
        include=[
            "object",
            "category",
            "bool"
        ]
    )
    .columns
    .tolist()

)

print(
    "Numerical features:",
    len(numeric_features)
)

print(
    numeric_features
)

print(
    "\nCategorical features:",
    len(categorical_features)
)

```

```

print(
    categorical_features
)

print(
    "\nExcluded columns:"
)

print(
    sorted(
        excluded_columns
    )
)

```

Numerical features: 8

```
['latitude', 'longitude', 'bathrooms', 'bedrooms', 'floorAreaSqM',
'livingRooms', 'history_percentageChange', 'history_numericChange']
```

Categorical features: 6

```
['country', 'outcode', 'tenure', 'propertyType', 'currentEnergyRating',
'history_date']
```

Excluded columns:

```
['fullAddress', 'history_price', 'postcode', 'rentEstimate_currentPrice',
'rentEstimate_lowerPrice', 'rentEstimate_upperPrice',
'saleEstimate_confidenceLevel', 'saleEstimate_currentPrice',
'saleEstimate_ingestedAt', 'saleEstimate_lowerPrice', 'saleEstimate_upperPrice',
'saleEstimate_valueChange.numericChange',
'saleEstimate_valueChange.percentageChange',
'saleEstimate_valueChange.saleDate']
```

## 1.12 12. Data Representation

The assignment requires the transformation from raw data into a computational representation to be made explicit.

The representation changes through the following stages:

CSV → DataFrame → Selected Features → Numerical Feature Matrix → Model Input

For one property:

$$x_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T \in \mathbb{R}^d$$

For the complete dataset:

$$X \in \mathbb{R}^{N \times d}$$

### 1.12.1 Questions

1. What does one row represent?
2. What does one feature dimension represent?
3. How are categorical features converted into numerical dimensions?
4. Why can the final model dimension be larger than the original dimension?

### 1.12.2 Answer

One row represents one property transaction.

Each original dimension corresponds to one selected property characteristic.

Categorical features are expanded into binary dimensions through one-hot encoding. Therefore, one categorical column can become several model-input dimensions.

```
[17]: # =====
# 12. Original data representation
# =====

N = len(X_raw)

d_original = X_raw.shape[1]

print(
    "Number of observations:",
    N
)

print(
    "Original feature dimension:",
    d_original
)

print(
    f"\nX ∈ ℝ^{N} × {d_original}"
)

print(
    "\nOne original property record:"
)

display(
```

```
X_raw.iloc[
    [0]
]
```

Number of observations: 418194

Original feature dimension: 14

X  $R^{(418194 \times 14)}$

One original property record:

|   | country | outcode | latitude  | longitude | bathrooms | bedrooms | floorAreaSqM | \ |
|---|---------|---------|-----------|-----------|-----------|----------|--------------|---|
| 0 | England | EC4A    | 51.517282 | -0.110314 | 1.0       | 1.0      | 45.0         |   |

|   | livingRooms | tenure    | propertyType       | currentEnergyRating | \   |
|---|-------------|-----------|--------------------|---------------------|-----|
| 0 | 1.0         | Leasehold | Purpose Built Flat |                     | NaN |

|   | history_date | history_percentageChange | history_numericChange |
|---|--------------|--------------------------|-----------------------|
| 0 | 1995-01-02   |                          | NaN                   |

## 1.13 13. Feature Engineering

### 1.13.1 Question

Can existing information be transformed into a representation that is more useful for regression without introducing future information?

Two transformations are considered.

First, a transaction date is decomposed into year and month when such a date is available.

Second, a logarithmic version of floor area is created to provide an additional representation for a potentially skewed numerical feature.

Current market estimates remain excluded because they may contain information from a later point in time than the historical transaction.

```
[18]: # =====
# 13. Feature engineering
# =====

X_engineered = X_raw.copy()

# -----
# Transform date columns into year and month
# -----

date_columns = [
```

```

column

for column in X_engineered.columns

    if "date" in column.lower()

]

new_date_features = []

for column in date_columns:

    parsed_date = pd.to_datetime(

        X_engineered[column],

        errors="coerce"

    )

    if parsed_date.notna().any():

        year_name = (

            f"{column}_year"

        )

        month_name = (

            f"{column}_month"

        )

        X_engineered[

            year_name

        ] = parsed_date.dt.year

        X_engineered[

            month_name

        ] = parsed_date.dt.month

        new_date_features.extend([

            year_name,

```

```

        month_name

    ])

    X_engineered = (
        X_engineered
        .drop(
            columns=[column]
        )
    )

# -----
# Log-transform property area
# -----

if "floorAreaSqM" in X_engineered.columns:

    X_engineered[
        "log_floorAreaSqM"
    ] = np.log1p(

        X_engineered[
            "floorAreaSqM"
        ].clip(
            lower=0
        )

    )

print(
    "Shape before engineering:",
    X_raw.shape
)

print(
    "Shape after engineering :",
    X_engineered.shape
)

new_features = [

    column

```

```

    for column in X_engineered.columns

    if column not in X_raw.columns

]

print(
    "\nCreated features:"
)

print(
    new_features
)

```

Shape before engineering: (418194, 14)

Shape after engineering : (418194, 16)

Created features:

['history\_date\_year', 'history\_date\_month', 'log\_floorAreaSqM']

## 1.14 14. Train, Validation, and Test Split

### 1.14.1 Question

How can model development be performed without allowing test observations to influence preprocessing or model selection?

The data are divided into:

$$D = D_{\text{train}} \cup D_{\text{validation}} \cup D_{\text{test}}$$

using a 70/15/15 split.

The training set is used to learn model and preprocessing parameters.

The validation set is used for model comparison and hyperparameter experiments.

The test set remains isolated until final evaluation.

Stratification is not used because house price is a continuous regression target.

```

[19]: # =====
# 14. 70 / 15 / 15 train-validation-test split
# =====

from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = (

```

```

train_test_split(

    X_engineered,
    y,

    test_size=0.30,

    random_state=RANDOM_SEED

)

)

X_val, X_test, y_val, y_test = (

    train_test_split(

        X_temp,
        y_temp,

        test_size=0.50,

        random_state=RANDOM_SEED

    )

)

print(
    "Training set :",
    X_train.shape,
    y_train.shape
)

print(
    "Validation set :",
    X_val.shape,
    y_val.shape
)

print(
    "Test set      :",
    X_test.shape,
    y_test.shape
)

```

```
Training set    : (292735, 16) (292735,)
Validation set  : (62729, 16) (62729,)
Test set       : (62730, 16) (62730,)
```

## 1.15 15. Preprocessing Pipeline

Different feature types require different transformations.

### 1.15.1 Numerical features

Median Imputation → Standardization

### 1.15.2 Categorical features

Most-Frequent Imputation → One-Hot Encoding

### 1.15.3 Question

How can preprocessing be applied consistently without causing data leakage?

### 1.15.4 Answer

The preprocessing transformer is fitted only on the training set.

The learned transformation is then applied to validation, test, and future unseen data.

Therefore, validation and test observations do not influence the learned imputation values, scaling parameters, or category mapping.

```
[20]: # =====
# 15. Build preprocessing pipeline
# =====

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler,
    OneHotEncoder
)

numeric_features = (
    X_train
    .select_dtypes(
        include=np.number
    )
    .columns
    .tolist()
```

```

)

categorical_features = (
    X_train
    .select_dtypes(
        include=[
            "object",
            "category",
            "bool"
        ]
    )
    .columns
    .tolist()
)

numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="median"
        )
    ),
    (
        "scaler",
        StandardScaler()
    )
])

categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (

```

```

        "onehot",
        OneHotEncoder(
            handle_unknown="ignore",
            min_frequency=20,
            sparse_output=False
        )
    )
])

preprocessor = ColumnTransformer(

    transformers=[

        (
            "numeric",
            numeric_pipeline,
            numeric_features
        ),

        (
            "categorical",
            categorical_pipeline,
            categorical_features
        )

    ],

    remainder="drop"
)

print(
    "Numerical feature count:",
    len(numeric_features)
)

print(
    "Categorical feature count:",
    len(categorical_features)
)

```

```

Numerical feature count: 11
Categorical feature count: 5

```

```

[21]: # =====
# Fit preprocessing on training data only
# =====

preprocessor.fit(
    X_train
)

X_train_processed = (
    preprocessor
    .transform(
        X_train
    )
)

X_val_processed = (
    preprocessor
    .transform(
        X_val
    )
)

X_test_processed = (
    preprocessor
    .transform(
        X_test
    )
)

feature_names = (
    preprocessor
    .get_feature_names_out()
)

d_final = len(
    feature_names
)

print(
    "Processed training shape  :",
    X_train_processed.shape
)

```

```

print(
    "Processed validation shape :",
    X_val_processed.shape
)

print(
    "Processed test shape      :",
    X_test_processed.shape
)

print(
    "\nOriginal feature dimension:",
    X_engineered.shape[1]
)

print(
    "Final model dimension      :",
    d_final
)

print(
    f"\nFinal model representation:"
)

print(
    f"X_train      "
    f"R^({X_train_processed.shape[0]} × {d_final})"
)

```

```

Processed training shape   : (292735, 205)
Processed validation shape : (62729, 205)
Processed test shape      : (62730, 205)

```

```

Original feature dimension: 16
Final model dimension      : 205

```

```

Final model representation:
X_train  R^(292735 × 205)

```

```

[22]: # =====
# Final numerical representation preview
# =====

processed_preview = pd.DataFrame(

    X_train_processed[:5],

```

```

    columns=feature_names
)

display(
    processed_preview
)

```

|   | numeric__latitude | numeric__longitude | numeric__bathrooms | \ |
|---|-------------------|--------------------|--------------------|---|
| 0 | -1.421310         | -0.487019          | -0.534874          |   |
| 1 | -0.674648         | -1.742785          | -0.534874          |   |
| 2 | 1.231512          | -0.398311          | 0.946285           |   |
| 3 | -0.834313         | 0.801719           | -0.534874          |   |
| 4 | 0.776212          | -0.131862          | -0.534874          |   |

|   | numeric__bedrooms | numeric__floorAreaSqM | numeric__livingRooms | \ |
|---|-------------------|-----------------------|----------------------|---|
| 0 | -1.285827         | -0.791011             | -0.446993            |   |
| 1 | -0.385884         | 0.587372              | -0.446993            |   |
| 2 | -1.285827         | -0.882903             | -0.446993            |   |
| 3 | -0.385884         | -0.791011             | -0.446993            |   |
| 4 | -0.385884         | -0.331550             | -0.446993            |   |

|   | numeric__history_percentageChange | numeric__history_numericChange | \ |
|---|-----------------------------------|--------------------------------|---|
| 0 | -0.121580                         | -0.112060                      |   |
| 1 | -0.125688                         | 0.243362                       |   |
| 2 | 0.635335                          | 0.161961                       |   |
| 3 | 0.355459                          | 0.339269                       |   |
| 4 | -0.316493                         | -0.284532                      |   |

|   | numeric__history_date_year | numeric__history_date_month | ... | \ |
|---|----------------------------|-----------------------------|-----|---|
| 0 | 0.950539                   | -0.785821                   | ... |   |
| 1 | 0.950539                   | -1.083351                   | ... |   |
| 2 | 1.057901                   | 1.594421                    | ... |   |
| 3 | 0.950539                   | 0.404300                    | ... |   |
| 4 | 1.057901                   | 0.701830                    | ... |   |

|   | categorical__propertyType_Terrace Property | \ |
|---|--------------------------------------------|---|
| 0 | 0.0                                        |   |
| 1 | 0.0                                        |   |
| 2 | 0.0                                        |   |
| 3 | 0.0                                        |   |
| 4 | 0.0                                        |   |

|   | categorical__propertyType_Terraced | \ |
|---|------------------------------------|---|
| 0 | 0.0                                |   |
| 1 | 0.0                                |   |
| 2 | 0.0                                |   |

|   |     |  |
|---|-----|--|
| 3 | 0.0 |  |
| 4 | 0.0 |  |

|   |                                               |  |
|---|-----------------------------------------------|--|
|   | categorical__propertyType_Terraced Bungalow \ |  |
| 0 | 0.0                                           |  |
| 1 | 0.0                                           |  |
| 2 | 0.0                                           |  |
| 3 | 0.0                                           |  |
| 4 | 0.0                                           |  |

|   |                                    |                                      |
|---|------------------------------------|--------------------------------------|
|   | categorical__currentEnergyRating_A | categorical__currentEnergyRating_B \ |
| 0 | 0.0                                | 0.0                                  |
| 1 | 0.0                                | 0.0                                  |
| 2 | 0.0                                | 0.0                                  |
| 3 | 0.0                                | 0.0                                  |
| 4 | 0.0                                | 0.0                                  |

|   |                                    |                                      |
|---|------------------------------------|--------------------------------------|
|   | categorical__currentEnergyRating_C | categorical__currentEnergyRating_D \ |
| 0 | 1.0                                | 0.0                                  |
| 1 | 0.0                                | 1.0                                  |
| 2 | 1.0                                | 0.0                                  |
| 3 | 0.0                                | 1.0                                  |
| 4 | 0.0                                | 1.0                                  |

|   |                                    |                                      |
|---|------------------------------------|--------------------------------------|
|   | categorical__currentEnergyRating_E | categorical__currentEnergyRating_F \ |
| 0 | 0.0                                | 0.0                                  |
| 1 | 0.0                                | 0.0                                  |
| 2 | 0.0                                | 0.0                                  |
| 3 | 0.0                                | 0.0                                  |
| 4 | 0.0                                | 0.0                                  |

|   |                                    |  |
|---|------------------------------------|--|
|   | categorical__currentEnergyRating_G |  |
| 0 | 0.0                                |  |
| 1 | 0.0                                |  |
| 2 | 0.0                                |  |
| 3 | 0.0                                |  |
| 4 | 0.0                                |  |

[5 rows x 205 columns]

## 1.16 16. Baseline Model

### 1.16.1 Question

How well can house price be predicted without learning relationships between property features and price?

A median baseline predicts the median training-set price for every validation property.

This baseline provides a reference point. The machine-learning models should achieve substantially lower error and a higher  $R^2$  than this non-learning strategy.

```
[23]: # =====  
# 16. Median baseline  
# =====  
  
from sklearn.dummy import DummyRegressor  
  
from sklearn.metrics import (  
    mean_absolute_error,  
    mean_squared_error,  
    r2_score  
)  
  
baseline_model = DummyRegressor(  
    strategy="median"  
)  
  
start_time = time.perf_counter()  
  
baseline_model.fit(  
    X_train_processed,  
    y_train  
)  
  
baseline_time = (  
    time.perf_counter()  
    -  
    start_time  
)  
  
baseline_prediction = (  
    baseline_model  
    .predict(  
        X_val_processed
```

```

    )
)

baseline_results = pd.DataFrame([

    "Model":
        "Median Baseline",

    "MAE":
        mean_absolute_error(
            y_val,
            baseline_prediction
        ),

    "RMSE":
        np.sqrt(
            mean_squared_error(
                y_val,
                baseline_prediction
            )
        ),

    "R2":
        r2_score(
            y_val,
            baseline_prediction
        ),

    "Training Time (s)":
        round(
            baseline_time,
            4
        )

])

display(
    baseline_results
)

```

|   | Model           | MAE           | RMSE         | R2        | Training Time (s) |
|---|-----------------|---------------|--------------|-----------|-------------------|
| 0 | Median Baseline | 414395.709289 | 1.359839e+06 | -0.031593 | 0.0044            |

## 1.17 17. Model Training

Five regression algorithms are compared.

1. Linear Regression
2. Ridge Regression
3. Decision Tree Regressor
4. Random Forest Regressor
5. Gradient Boosting Regressor

### 1.17.1 Question

Why compare different model families instead of training only one algorithm?

### 1.17.2 Answer

Different models represent different assumptions about the relationship between property features and price.

Linear Regression provides a simple linear baseline.

Ridge adds regularization to control coefficient magnitude.

Decision Tree models nonlinear decision rules.

Random Forest combines multiple trees.

Gradient Boosting builds an ensemble sequentially to improve prediction accuracy.

```
[24]: # =====  
# 17. Define the five regression models  
# =====  
  
from sklearn.linear_model import (  
    LinearRegression,  
    Ridge  
)  
  
from sklearn.tree import (  
    DecisionTreeRegressor  
)  
  
from sklearn.ensemble import (  
    RandomForestRegressor,  
    GradientBoostingRegressor  
)  
  
models = {  
    "Linear Regression":  
        LinearRegression(),
```

```

    "Ridge":
        Ridge(
            alpha=1.0
        ),

    "Decision Tree":
        DecisionTreeRegressor(
            max_depth=12,
            min_samples_leaf=5,
            random_state=RANDOM_SEED
        ),

    "Random Forest":
        RandomForestRegressor(
            n_estimators=200,
            max_depth=16,
            min_samples_leaf=2,
            n_jobs=-1,
            random_state=RANDOM_SEED
        ),

    "Gradient Boosting":
        GradientBoostingRegressor(
            n_estimators=200,
            learning_rate=0.05,
            max_depth=4,
            random_state=RANDOM_SEED
        )
}

print(
    "Models prepared:"
)

for model_name in models:

    print(
        "-",
        model_name
    )

```

Models prepared:

- Linear Regression
- Ridge
- Decision Tree

- Random Forest
- Gradient Boosting

```
[25]: # =====  
# Train the five models  
# =====  
  
model_results = []  
  
trained_models = {}  
  
validation_predictions = {}  
  
for model_name, model in models.items():  
  
    print(  
        f"Training {model_name}..."  
    )  
  
    start_time = (  
        time.perf_counter()  
    )  
  
    model.fit(  
  
        X_train_processed,  
  
        y_train  
    )  
  
    training_time = (  
  
        time.perf_counter()  
        -  
        start_time  
    )  
  
    prediction = (  
  
        model
```

```

        .predict(
            X_val_processed
        )

    )

    model_results.append({

        "Model":
            model_name,

        "MAE":
            mean_absolute_error(
                y_val,
                prediction
            ),

        "RMSE":
            np.sqrt(
                mean_squared_error(
                    y_val,
                    prediction
                )
            ),

        "R2":
            r2_score(
                y_val,
                prediction
            ),

        "Training Time (s)":
            round(
                training_time,
                4
            )

    })

    trained_models[
        model_name
    ] = model

    validation_predictions[

```

```

        model_name
    ] = prediction

comparison_df = (

    pd.DataFrame(
        model_results
    )

    .sort_values(
        "RMSE"
    )

    .reset_index(
        drop=True
    )

)

display(
    comparison_df
)

```

Training Linear Regression...  
 Training Ridge...  
 Training Decision Tree...  
 Training Random Forest...  
 Training Gradient Boosting...

|   | Model             | MAE           | RMSE         | R2       | Training Time (s) |
|---|-------------------|---------------|--------------|----------|-------------------|
| 0 | Random Forest     | 152731.198848 | 8.619428e+05 | 0.585533 | 485.7822          |
| 1 | Gradient Boosting | 184571.644547 | 8.876463e+05 | 0.560445 | 749.3185          |
| 2 | Decision Tree     | 197778.677719 | 9.531536e+05 | 0.493174 | 8.5116            |
| 3 | Linear Regression | 294794.548124 | 1.078648e+06 | 0.350928 | 2.0141            |
| 4 | Ridge             | 294747.888309 | 1.078660e+06 | 0.350914 | 0.4980            |

## 1.18 18. Model Comparison

### 1.18.1 Question

Which regression model provides the most reliable prediction of house price?

The models are compared using five complementary metrics:

- Mean Absolute Error
- Mean Squared Error
- Root Mean Squared Error
- $R^2$

- Mean Absolute Percentage Error

Each metric captures a different aspect of regression performance.

MAE measures the average absolute prediction error.

MSE gives greater penalty to large errors.

RMSE is expressed in the same unit as house price and is particularly sensitive to large prediction errors.

$R^2$  measures how much of the variation in house price is explained by the model.

MAPE measures the average prediction error relative to the actual price.

Lower MAE, MSE, RMSE, and MAPE are better, while higher  $R^2$  is better.

The models are first compared on the validation set. The validation set is used for model comparison and hyperparameter experiments, while the test set remains isolated until final evaluation.

```
[26]: # =====
# 18. Model Comparison
# =====

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error
)

model_results = []

trained_models = {}

validation_predictions = {}

for model_name, model in models.items():

    print(
        f"Training {model_name}..."
    )

    start_time = time.perf_counter()

    model.fit(
        X_train_processed,
        y_train
    )
```

```

training_time = (
    time.perf_counter()
    - start_time
)

prediction = model.predict(
    X_val_processed
)

mae = mean_absolute_error(
    y_val,
    prediction
)

mse = mean_squared_error(
    y_val,
    prediction
)

rmse = np.sqrt(
    mse
)

r2 = r2_score(
    y_val,
    prediction
)

mape = (
    mean_absolute_percentage_error(
        y_val,
        prediction
    )
    * 100
)

model_results.append({
    "Model":
        model_name,

    "MAE":
        mae,

    "MSE":
        mse,

```

```

        "RMSE":
            rmse,

        "R2":
            r2,

        "MAPE (%)":
            mape,

        "Training Time (s)":
            training_time

    })

    trained_models[
        model_name
    ] = model

    validation_predictions[
        model_name
    ] = prediction

comparison_df = (
    pd.DataFrame(
        model_results
    )
    .sort_values(
        "RMSE"
    )
    .reset_index(
        drop=True
    )
)

display(
    comparison_df.round(4)
)

```

Training Linear Regression...  
 Training Ridge...  
 Training Decision Tree...  
 Training Random Forest...  
 Training Gradient Boosting...

| Model | MAE | MSE | RMSE | R2 \ |
|-------|-----|-----|------|------|
|-------|-----|-----|------|------|

|   |                   |             |              |              |        |
|---|-------------------|-------------|--------------|--------------|--------|
| 0 | Random Forest     | 152731.1988 | 7.429454e+11 | 8.619428e+05 | 0.5855 |
| 1 | Gradient Boosting | 184571.6445 | 7.879160e+11 | 8.876463e+05 | 0.5604 |
| 2 | Decision Tree     | 197778.6777 | 9.085018e+11 | 9.531536e+05 | 0.4932 |
| 3 | Linear Regression | 294794.5481 | 1.163482e+12 | 1.078648e+06 | 0.3509 |
| 4 | Ridge             | 294747.8883 | 1.163507e+12 | 1.078660e+06 | 0.3509 |

|   | MAPE (%) | Training Time (s) |
|---|----------|-------------------|
| 0 | 34.6867  | 486.0140          |
| 1 | 41.2625  | 703.2105          |
| 2 | 41.6170  | 8.6669            |
| 3 | 93.1496  | 2.1490            |
| 4 | 93.1155  | 0.5335            |

### 1.18.2 Interpretation

The comparison table provides a broader view of model behaviour than a single metric.

A model with the lowest RMSE is not automatically the best in every respect. Its MAE, MAPE,  $R^2$ , and training cost should also be considered.

This is especially important for house-price prediction because a model may achieve a good average error while still making very large errors for a smaller number of expensive properties.

```
[27]: # =====
# Compare models across five regression metrics
# =====

metrics_for_plot = [
    "MAE",
    "MSE",
    "RMSE",
    "R2",
    "MAPE (%)"
]

for metric in metrics_for_plot:

    ascending = (
        metric != "R2"
    )

    plot_df = (
        comparison_df
        .sort_values(
            metric,
            ascending=ascending
        )
    )
```

```

plt.figure(
    figsize=(9, 5)
)

plt.barh(
    plot_df["Model"],
    plot_df[metric]
)

plt.xlabel(
    metric
)

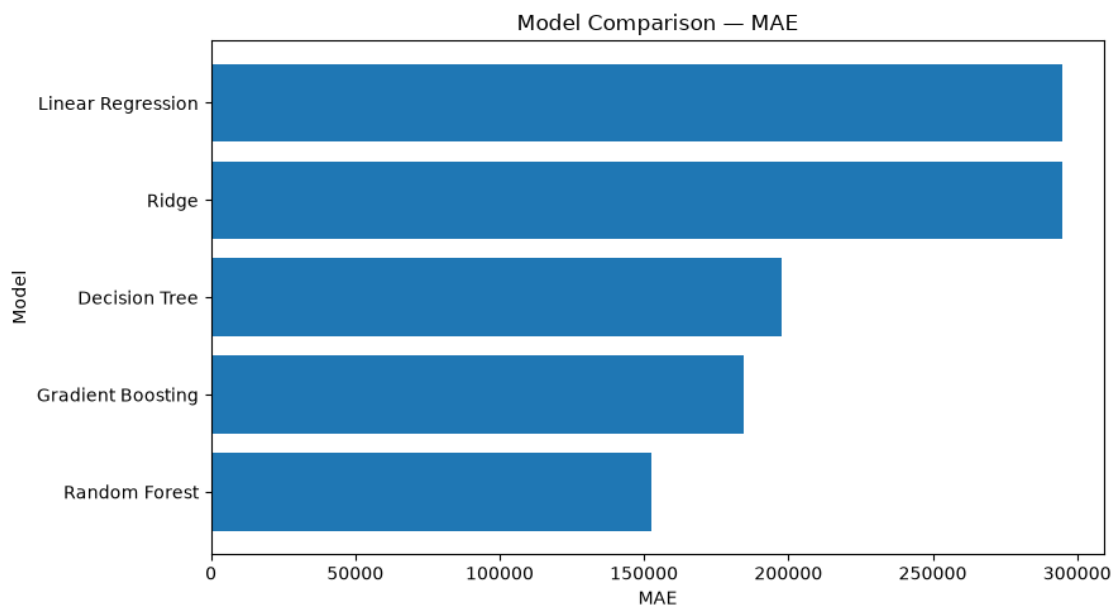
plt.ylabel(
    "Model"
)

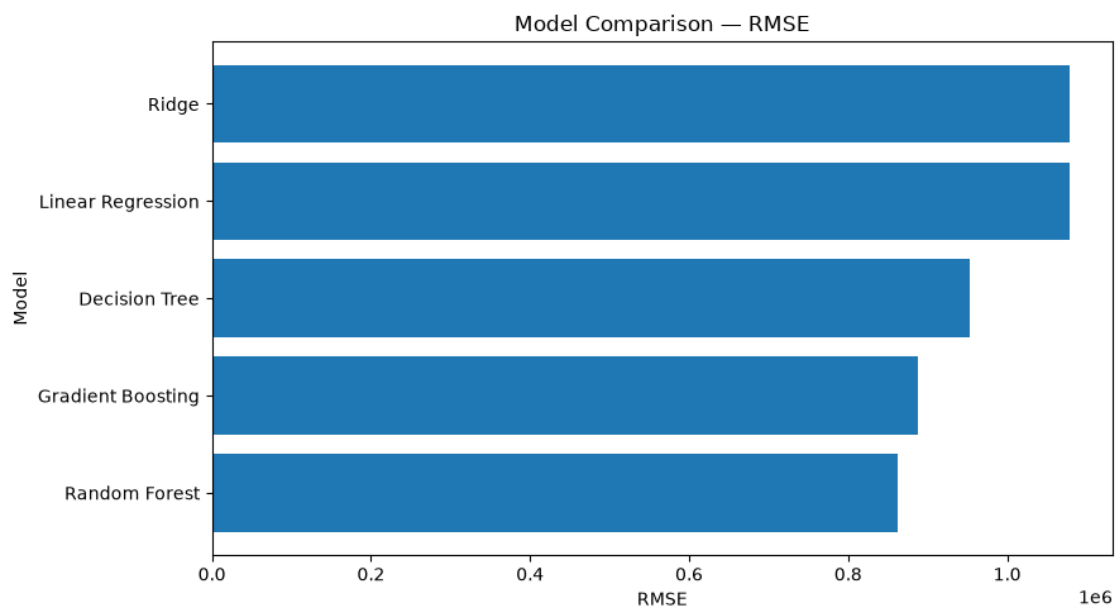
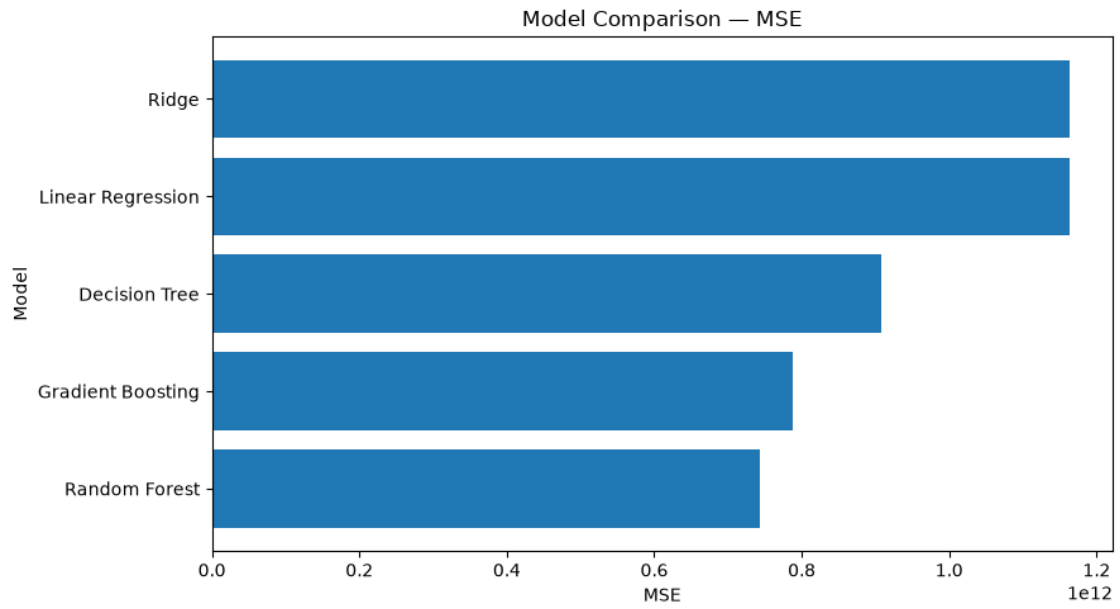
plt.title(
    f"Model Comparison - {metric}"
)

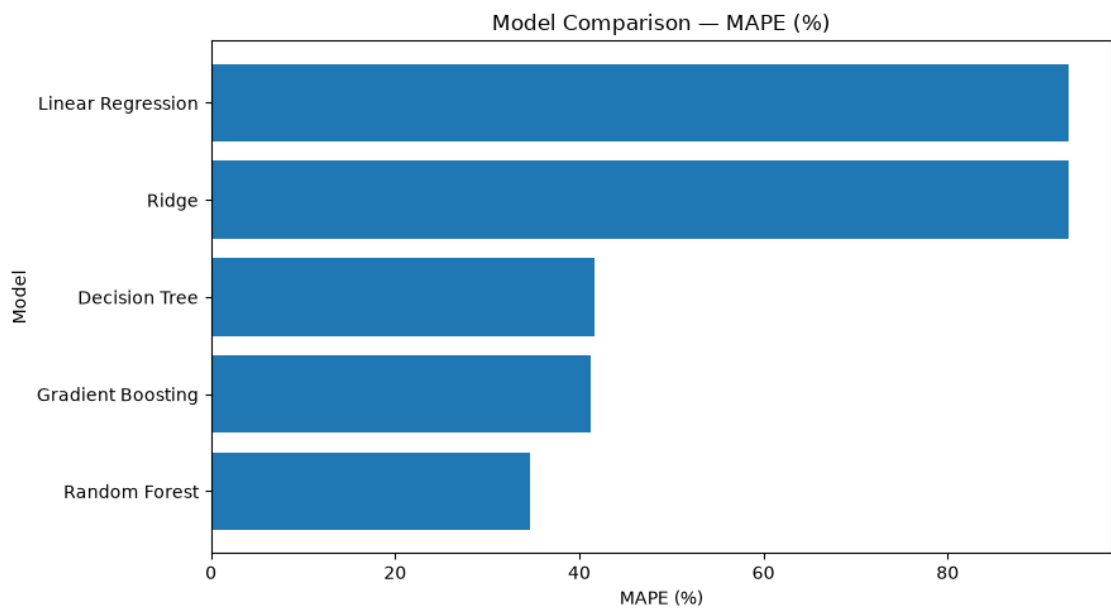
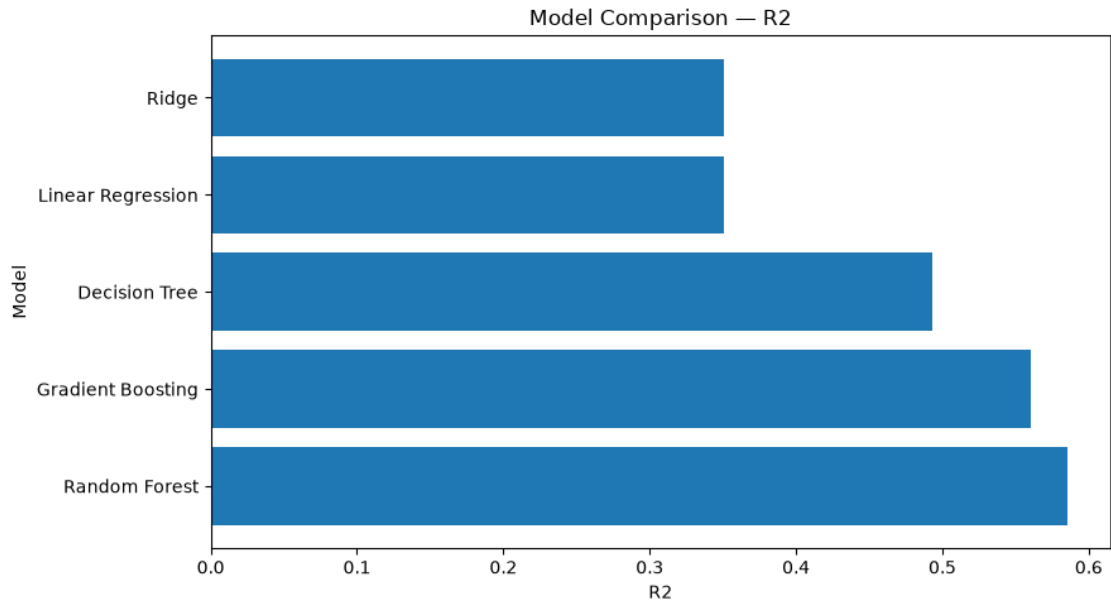
plt.tight_layout()

plt.show()

```







## 1.19 19. Evaluation

### 1.19.1 Question

How accurately can the selected model predict house prices on previously unseen data?

The selected model is evaluated on the isolated test set using the same five regression metrics:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Lower MAE, MSE, RMSE, and MAPE indicate smaller prediction errors.

Higher  $R^2$  indicates stronger explanatory performance.

The test set is used only after model selection so that it remains an unbiased estimate of final generalization performance.

```
[28]: # =====
# 19. Validation Evaluation
# =====

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error
)

# -----
# Evaluate the five trained models on the validation set
# -----

evaluation_results = []

validation_predictions = {}

for model_name, model in trained_models.items():

    # -----
```

```

# Generate validation predictions
# -----

prediction = model.predict(
    X_val_processed
)

# -----
# Store predictions for later analysis
# -----

validation_predictions[
    model_name
] = prediction

# -----
# Calculate evaluation metrics
# -----

mae = mean_absolute_error(
    y_val,
    prediction
)

mse = mean_squared_error(
    y_val,
    prediction
)

rmse = np.sqrt(
    mse
)

r2 = r2_score(
    y_val,
    prediction
)

mape = (
    mean_absolute_percentage_error(
        y_val,

```

```

        prediction
    )
    * 100
)

# -----
# Store results
# -----

evaluation_results.append({

    "Model":
        model_name,

    "MAE":
        mae,

    "MSE":
        mse,

    "RMSE":
        rmse,

    "R2":
        r2,

    "MAPE (%)":
        mape

})

# -----
# Create evaluation table
# -----

evaluation_df = (

    pd.DataFrame(
        evaluation_results
    )

    .sort_values(
        by="RMSE",
        ascending=True
    )
)

```

```

        .reset_index(
            drop=True
        )
    )

# -----
# Display results
# -----

display(
    evaluation_df.round(4)
)

```

|   | Model             | MAE         | MSE          | RMSE         | R2     | \ |
|---|-------------------|-------------|--------------|--------------|--------|---|
| 0 | Random Forest     | 152731.1988 | 7.429454e+11 | 8.619428e+05 | 0.5855 |   |
| 1 | Gradient Boosting | 184571.6445 | 7.879160e+11 | 8.876463e+05 | 0.5604 |   |
| 2 | Decision Tree     | 197778.6777 | 9.085018e+11 | 9.531536e+05 | 0.4932 |   |
| 3 | Linear Regression | 294794.5481 | 1.163482e+12 | 1.078648e+06 | 0.3509 |   |
| 4 | Ridge             | 294747.8883 | 1.163507e+12 | 1.078660e+06 | 0.3509 |   |

|   | MAPE (%) |
|---|----------|
| 0 | 34.6867  |
| 1 | 41.2625  |
| 2 | 41.6170  |
| 3 | 93.1496  |
| 4 | 93.1155  |

```

[29]: # =====
# Validation performance comparison
# =====

metrics_for_plot = [
    "MAE",
    "MSE",
    "RMSE",
    "R2",
    "MAPE (%)"
]

for metric in metrics_for_plot:

    ascending = (
        metric != "R2"
    )

```

```

)

plot_df = (
    evaluation_df
    .sort_values(
        by=metric,
        ascending=ascending
    )
)

plt.figure(
    figsize=(9, 5)
)

plt.barh(
    plot_df["Model"],
    plot_df[metric]
)

plt.xlabel(
    metric
)

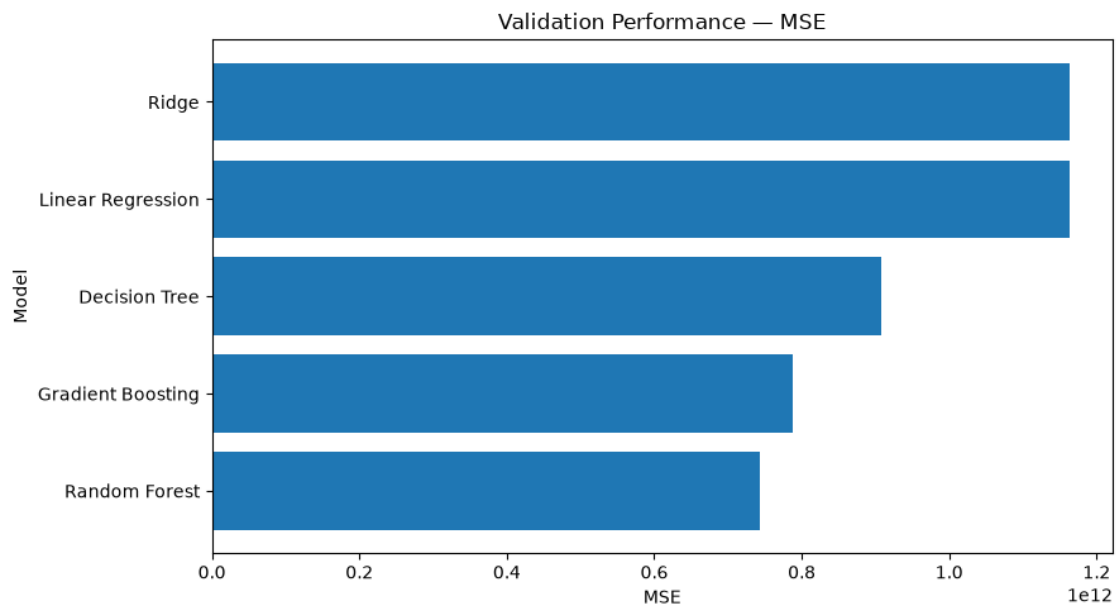
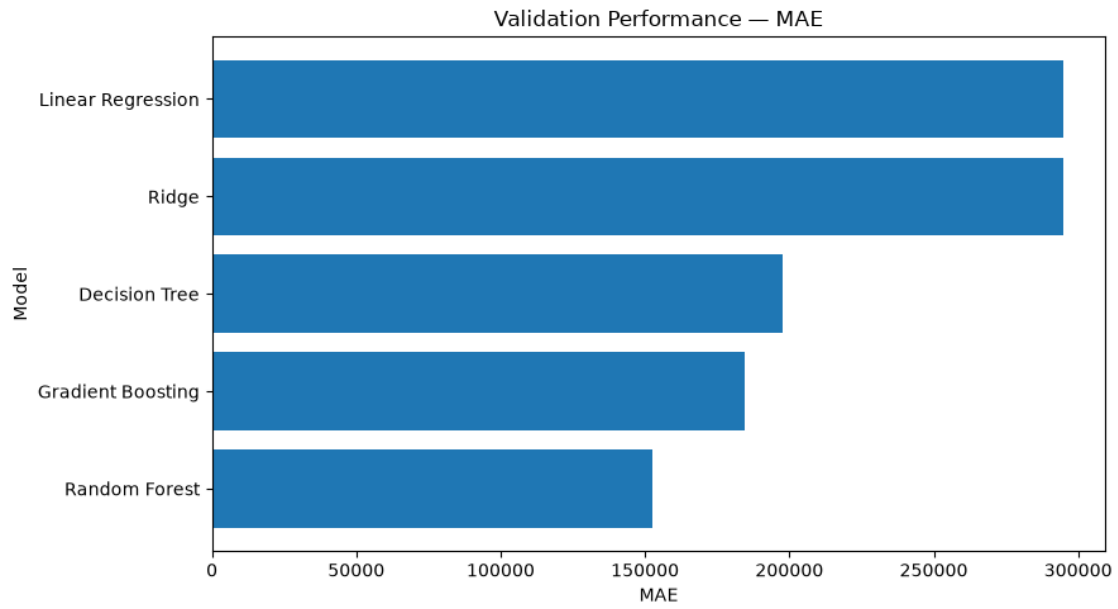
plt.ylabel(
    "Model"
)

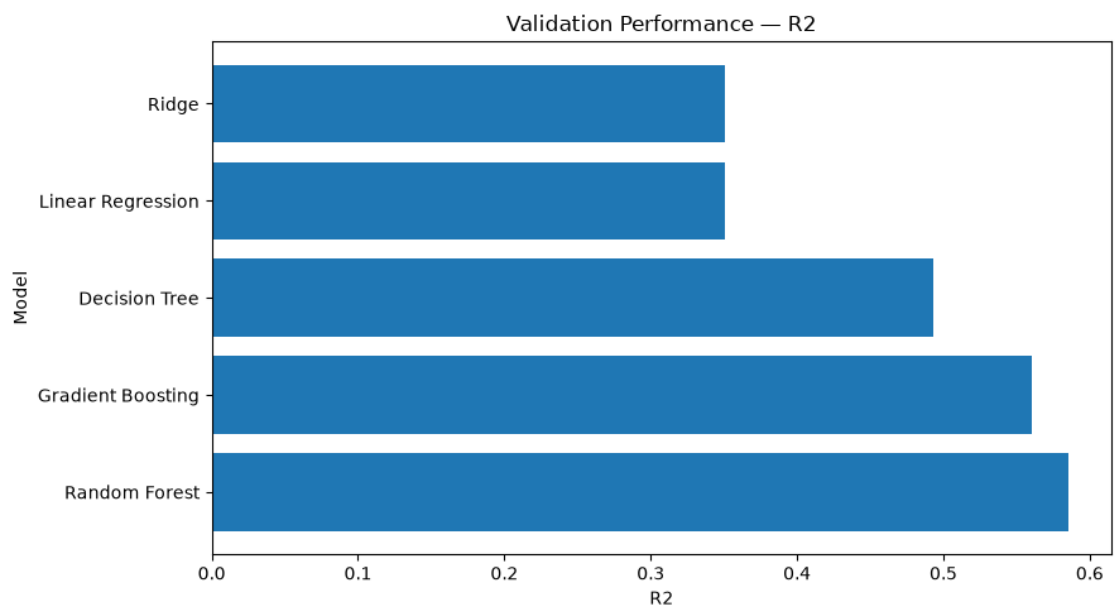
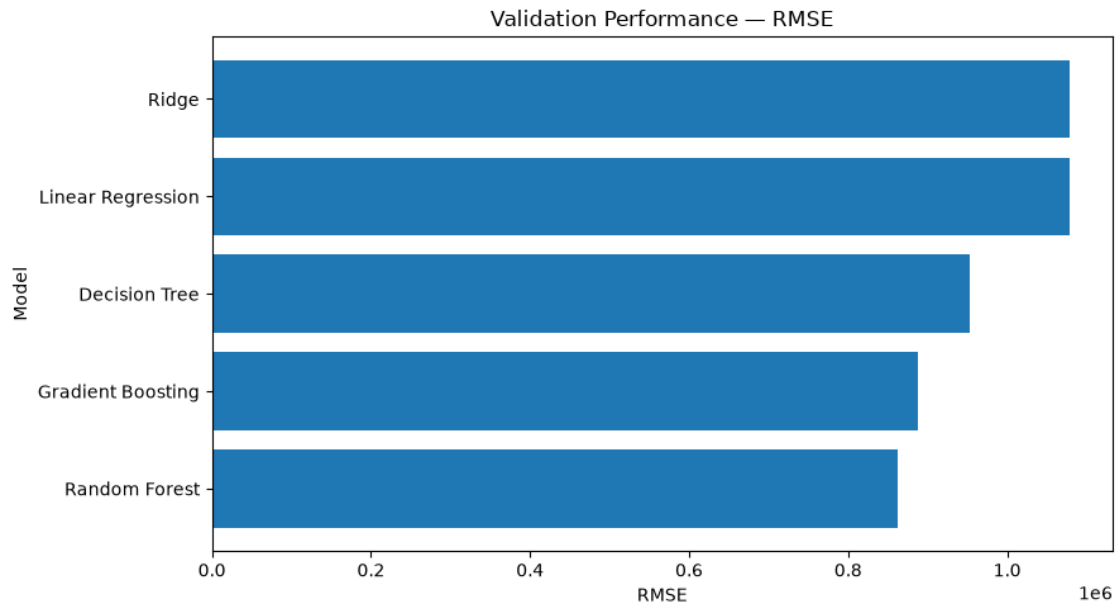
plt.title(
    f"Validation Performance - {metric}"
)

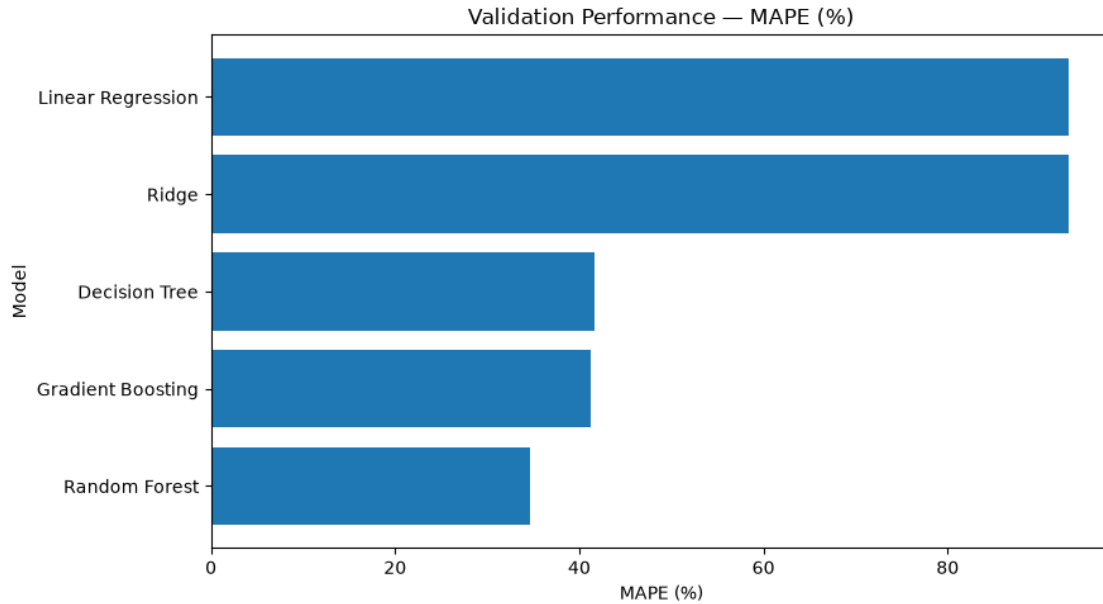
plt.tight_layout()

plt.show()

```







### 1.19.2 Interpretation

The largest absolute errors identify properties for which the final model has the greatest difficulty estimating price.

These errors should not automatically be treated as data errors. They may result from unusual property characteristics, extreme prices, location-specific effects, or information that is not represented by the selected features.

This analysis therefore provides evidence about the model's limitations rather than simply reporting another performance score.

## 1.20 20. Hyperparameter Experiments

Three focused experiments are used to investigate how selected hyperparameters affect validation performance.

The experiments are intentionally limited to parameters with a clear modelling interpretation.

The validation set is used for these experiments, while the test set remains untouched.

### 1.20.1 Experiment 1 — Ridge Regularization

**Question** How does regularization strength affect Ridge Regression?

The parameter  $\alpha$  controls the strength of regularization.

The following values are compared:

$$\alpha \in \{0.01, 0.1, 1, 10, 100\}$$

The objective is to determine whether regularization improves validation performance compared with ordinary Linear Regression.

```
[30]: # =====  
# Experiment 1: Ridge alpha  
# =====  
  
ridge_results = []  
  
for alpha in [  
    0.01,  
    0.1,  
    1,  
    10,  
    100  
]:  
  
    model = Ridge(  
        alpha=alpha  
    )  
  
    start_time = (  
        time.perf_counter()  
    )  
  
    model.fit(  
  
        X_train_processed,  
  
        y_train  
    )  
  
    training_time = (  
  
        time.perf_counter()  
        -  
        start_time  
    )  
  
    prediction = (  

```

```

        model
        .predict(
            X_val_processed
        )
    )

ridge_results.append({

    "alpha":
        alpha,

    "MAE":
        mean_absolute_error(
            y_val,
            prediction
        ),

    "RMSE":
        np.sqrt(
            mean_squared_error(
                y_val,
                prediction
            )
        ),

    "R2":
        r2_score(
            y_val,
            prediction
        ),

    "Training Time (s)":
        round(
            training_time,
            4
        )

    })

ridge_experiment_df = (

    pd.DataFrame(
        ridge_results

```

```

    )

    .sort_values(
        "RMSE"
    )

    .reset_index(
        drop=True
    )
)

display(
    ridge_experiment_df
)

```

|   | alpha  | MAE           | RMSE         | R2       | Training Time (s) |
|---|--------|---------------|--------------|----------|-------------------|
| 0 | 0.01   | 294794.079248 | 1.078648e+06 | 0.350928 | 0.4237            |
| 1 | 0.10   | 294789.861381 | 1.078649e+06 | 0.350927 | 0.4355            |
| 2 | 1.00   | 294747.888309 | 1.078660e+06 | 0.350914 | 0.4318            |
| 3 | 10.00  | 294349.663503 | 1.078775e+06 | 0.350775 | 0.4563            |
| 4 | 100.00 | 291268.501383 | 1.080151e+06 | 0.349118 | 0.4067            |

### 1.20.2 Experiment 2 — Random Forest Ensemble Size

**Question** Does increasing the number of trees improve predictive performance enough to justify the additional computational cost?

The experiment evaluates:

$$n_{\text{estimators}} \in \{50, 100, 200, 300\}$$

All other Random Forest parameters remain fixed.

```

[31]: # =====
# Experiment 2: Random Forest number of trees
# =====

rf_results = []

for n_trees in [
    50,
    100,
    200,
    300
]:

```

```

model = RandomForestRegressor(

    n_estimators=n_trees,

    max_depth=16,

    min_samples_leaf=2,

    n_jobs=-1,

    random_state=RANDOM_SEED

)

start_time = (
    time.perf_counter()
)

model.fit(

    X_train_processed,

    y_train

)

training_time = (

    time.perf_counter()
    -
    start_time

)

prediction = (

    model
    .predict(
        X_val_processed
    )

)

```

```

rf_results.append({

    "n_estimators":
        n_trees,

    "MAE":
        mean_absolute_error(
            y_val,
            prediction
        ),

    "RMSE":
        np.sqrt(
            mean_squared_error(
                y_val,
                prediction
            )
        ),

    "R2":
        r2_score(
            y_val,
            prediction
        ),

    "Training Time (s)":
        round(
            training_time,
            4
        )

})

rf_experiment_df = (

    pd.DataFrame(
        rf_results
    )

    .sort_values(
        "RMSE"
    )

    .reset_index(

```

```

        drop=True
    )

)

display(
    rf_experiment_df
)

```

|   | n_estimators | MAE           | RMSE          | R2       | Training Time (s) |
|---|--------------|---------------|---------------|----------|-------------------|
| 0 | 50           | 152361.292843 | 854701.806253 | 0.592467 | 124.8755          |
| 1 | 100          | 152392.155141 | 857399.708406 | 0.589890 | 245.0910          |
| 2 | 300          | 152382.472572 | 861086.773907 | 0.586356 | 734.0384          |
| 3 | 200          | 152731.198848 | 861942.827236 | 0.585533 | 490.6894          |

### 1.20.3 Experiment 3 — Gradient Boosting Learning Rate

**Question** How does the learning rate affect the trade-off between prediction performance and training behaviour?

The experiment evaluates:

$$learning\_rate \in \{0.01, 0.05, 0.1, 0.2\}$$

The number of boosting stages and tree depth remain fixed.

```

[32]: # =====
# Experiment 3: Gradient Boosting learning rate
# =====

gb_results = []

for learning_rate in [
    0.01,
    0.05,
    0.1,
    0.2
]:

    model = GradientBoostingRegressor(

        n_estimators=200,

        learning_rate=learning_rate,

        max_depth=4,

```

```

        random_state=RANDOM_SEED
    )

    start_time = (
        time.perf_counter()
    )

    model.fit(

        X_train_processed,

        y_train

    )

    training_time = (

        time.perf_counter()
        -
        start_time

    )

    prediction = (

        model
        .predict(
            X_val_processed
        )

    )

    gb_results.append({

        "learning_rate":
            learning_rate,

        "MAE":
            mean_absolute_error(
                y_val,

```

```

        prediction
    ),

    "RMSE":
        np.sqrt(
            mean_squared_error(
                y_val,
                prediction
            )
        ),

    "R2":
        r2_score(
            y_val,
            prediction
        ),

    "Training Time (s)":
        round(
            training_time,
            4
        )

})

gb_experiment_df = (

    pd.DataFrame(
        gb_results
    )

    .sort_values(
        "RMSE"
    )

    .reset_index(
        drop=True
    )

)

display(
    gb_experiment_df
)

```

| learning_rate | MAE | RMSE | R2 | Training Time (s) |
|---------------|-----|------|----|-------------------|
|---------------|-----|------|----|-------------------|

|   |      |               |              |          |          |
|---|------|---------------|--------------|----------|----------|
| 0 | 0.20 | 170394.504560 | 8.211224e+05 | 0.623860 | 908.0307 |
| 1 | 0.10 | 175449.553588 | 8.526443e+05 | 0.594427 | 741.5697 |
| 2 | 0.05 | 184571.644547 | 8.876463e+05 | 0.560445 | 700.0012 |
| 3 | 0.01 | 259640.091686 | 1.042030e+06 | 0.394249 | 713.1424 |

## 1.21 21. Model Selection

### 1.21.1 Question

Which model should be selected for final deployment?

The final candidate is selected using the validation data only.

RMSE is the primary criterion because it places greater penalty on large price-estimation errors.

MAE,  $R^2$ , training time, model complexity, robustness, and deployment practicality are considered as supporting criteria.

The test set is not used during this decision.

```
[33]: # =====
# 21. Identify best configuration from each experiment
# =====

best_ridge_alpha = (
    ridge_experiment_df
    .iloc[0]["alpha"]
)

best_rf_trees = (
    rf_experiment_df
    .iloc[0]["n_estimators"]
)

best_gb_learning_rate = (
    gb_experiment_df
    .iloc[0]["learning_rate"]
)

print(
    "Best Ridge alpha:",
    best_ridge_alpha
)

print(
    "Best Random Forest n_estimators:",
    best_rf_trees
)
```

```
)

print(
    "Best Gradient Boosting learning rate:",
    best_gb_learning_rate
)
```

Best Ridge alpha: 0.01

Best Random Forest n\_estimators: 50.0

Best Gradient Boosting learning rate: 0.2

```
[34]: # =====
# Build tuned candidate models
# =====

tuned_candidates = {

    "Linear Regression":
        LinearRegression(),

    "Ridge Tuned":
        Ridge(
            alpha=best_ridge_alpha
        ),

    "Decision Tree":
        DecisionTreeRegressor(
            max_depth=12,
            min_samples_leaf=5,
            random_state=RANDOM_SEED
        ),

    "Random Forest Tuned":
        RandomForestRegressor(

            n_estimators=int(
                best_rf_trees
            ),

            max_depth=16,

            min_samples_leaf=2,

            n_jobs=-1,

            random_state=RANDOM_SEED
        )
}
```

```

    ),

    "Gradient Boosting Tuned":
        GradientBoostingRegressor(

            n_estimators=200,

            learning_rate=
                best_gb_learning_rate,

            max_depth=4,

            random_state=RANDOM_SEED

        )
}

tuned_results = []

tuned_models = {}

for model_name, model in tuned_candidates.items():

    start_time = (
        time.perf_counter()
    )

    model.fit(

        X_train_processed,

        y_train

    )

    training_time = (

        time.perf_counter()
        -
        start_time

    )

```

```

prediction = (

    model
    .predict(
        X_val_processed
    )

)

tuned_results.append({

    "Model":
        model_name,

    "MAE":
        mean_absolute_error(
            y_val,
            prediction
        ),

    "RMSE":
        np.sqrt(
            mean_squared_error(
                y_val,
                prediction
            )
        ),

    "R2":
        r2_score(
            y_val,
            prediction
        ),

    "Training Time (s)":
        round(
            training_time,
            4
        )

})

tuned_models[

```

```

        model_name
    ] = model

tuned_comparison_df = (

    pd.DataFrame(
        tuned_results
    )

    .sort_values(
        "RMSE"
    )

    .reset_index(
        drop=True
    )

)

display(
    tuned_comparison_df
)

```

|   | Model                   | MAE           | RMSE         | R2 \     |
|---|-------------------------|---------------|--------------|----------|
| 0 | Gradient Boosting Tuned | 170394.504560 | 8.211224e+05 | 0.623860 |
| 1 | Random Forest Tuned     | 152361.292843 | 8.547018e+05 | 0.592467 |
| 2 | Decision Tree           | 197778.677719 | 9.531536e+05 | 0.493174 |
| 3 | Linear Regression       | 294794.548124 | 1.078648e+06 | 0.350928 |
| 4 | Ridge Tuned             | 294794.079248 | 1.078648e+06 | 0.350928 |

|   | Training Time (s) |
|---|-------------------|
| 0 | 880.2323          |
| 1 | 149.7257          |
| 2 | 10.8623           |
| 3 | 2.9963            |
| 4 | 0.8954            |

### 1.21.2 Selection Decision

The model with the lowest validation RMSE is selected as the deployment candidate.

The final choice is based on validation performance because the test set must remain an unbiased source of final performance estimation.

The selected model will be evaluated on the test set only after this decision has been made.

## 1.22 22. Error Analysis

### 1.22.1 Question

Where does the selected model make its largest prediction errors?

For each property, the residual is:

$$e_i = y_i - \hat{y}_i$$

The absolute error is:

$$|e_i| = |y_i - \hat{y}_i|$$

Three views are used:

1. actual price versus predicted price;
2. residual distribution;
3. largest absolute errors.

### 1.22.2 Answer

These analyses help determine whether errors are centred around zero, whether the model systematically overestimates or underestimates prices, and whether errors become larger for expensive or unusual properties.

```
[35]: # =====  
# 22. Prepare validation predictions for error analysis  
# =====  
  
selected_model_name = (  
    tuned_comparison_df  
    .iloc[0]["Model"]  
)  
  
selected_model = (  
    tuned_models[  
        selected_model_name  
    ]  
)  
  
selected_validation_prediction = (  
    selected_model  
    .predict(  
        X_val_processed  
    )
```

```
)

print(
    "Selected model:",
    selected_model_name
)
```

Selected model: Gradient Boosting Tuned

```
[36]: # =====
# Actual vs predicted
# =====

plt.figure(
    figsize=(7, 7)
)

plt.scatter(

    y_val,

    selected_validation_prediction,

    alpha=0.25

)

lower = min(

    y_val.min(),

    selected_validation_prediction.min()

)

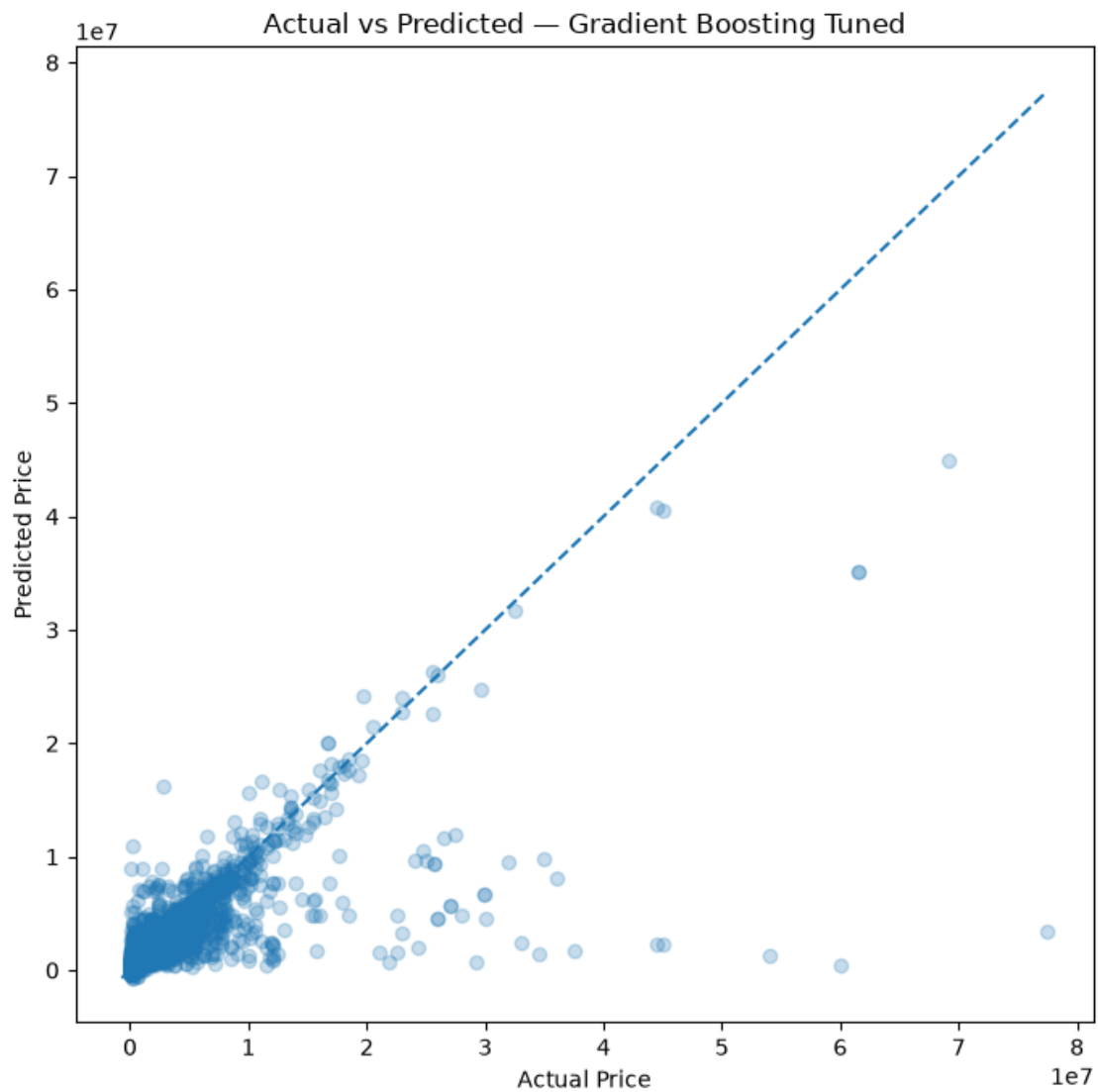
upper = max(

    y_val.max(),

    selected_validation_prediction.max()

)
```

```
plt.plot(  
    [lower, upper],  
    [lower, upper],  
    linestyle="--"  
)  
  
plt.xlabel(  
    "Actual Price"  
)  
  
plt.ylabel(  
    "Predicted Price"  
)  
  
plt.title(  
    f"Actual vs Predicted - {selected_model_name}"  
)  
  
plt.tight_layout()  
  
plt.show()
```



```
[37]: # =====  
# Residual distribution  
# =====  
  
residuals = (  
    y_val.to_numpy()  
    -  
    selected_validation_prediction  
)
```

```

plt.figure(
    figsize=(8, 4)
)

plt.hist(
    residuals,
    bins=50
)

plt.axvline(
    0,
    linestyle="--"
)

plt.xlabel(
    "Residual"
)

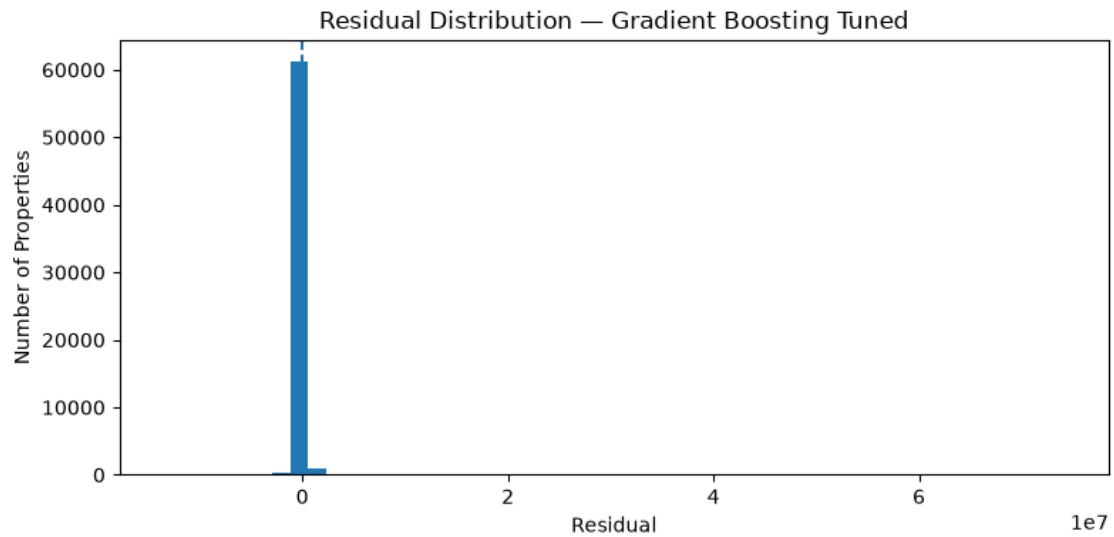
plt.ylabel(
    "Number of Properties"
)

plt.title(
    f"Residual Distribution - {selected_model_name}"
)

plt.tight_layout()

plt.show()

```



```
[38]: # =====
# Largest prediction errors
# =====

error_analysis_df = pd.DataFrame({

    "Actual Price":
        y_val.to_numpy(),

    "Predicted Price":
        selected_validation_prediction,

    "Residual":
        residuals,

    "Absolute Error":
        np.abs(residuals)

})

display(

    error_analysis_df

    .sort_values(
        "Absolute Error",
        ascending=False
    )
)
```

```
)

.head(10)

)
```

|       | Actual Price | Predicted Price | Residual     | Absolute Error |
|-------|--------------|-----------------|--------------|----------------|
| 27472 | 77500000     | 3.404110e+06    | 7.409589e+07 | 7.409589e+07   |
| 60072 | 60000000     | 3.722133e+05    | 5.962779e+07 | 5.962779e+07   |
| 40808 | 54000000     | 1.351699e+06    | 5.264830e+07 | 5.264830e+07   |
| 13809 | 45000000     | 2.287829e+06    | 4.271217e+07 | 4.271217e+07   |
| 46896 | 44500000     | 2.262281e+06    | 4.223772e+07 | 4.223772e+07   |
| 7553  | 37500000     | 1.641827e+06    | 3.585817e+07 | 3.585817e+07   |
| 52996 | 34500000     | 1.363616e+06    | 3.313638e+07 | 3.313638e+07   |
| 26749 | 33000000     | 2.364182e+06    | 3.063582e+07 | 3.063582e+07   |
| 17588 | 29160447     | 6.504100e+05    | 2.851004e+07 | 2.851004e+07   |
| 22659 | 36000000     | 8.162777e+06    | 2.783722e+07 | 2.783722e+07   |

### 1.22.3 Error Interpretation

The actual-versus-predicted plot is expected to concentrate around the diagonal when predictions are accurate.

Residuals centred around zero indicate limited systematic bias.

Large residuals are particularly important because they may correspond to high-value properties, unusual property characteristics, or locations that are not sufficiently represented by the available features.

## 1.23 23. Model Persistence

### 1.23.1 Question

How can the final machine-learning system be reused without retraining?

The final preprocessing transformer and regression model are stored together in a single pipeline.

This preserves:

- numerical imputation;
- numerical scaling;
- categorical imputation;
- one-hot encoding;
- feature ordering;
- trained model parameters.

The persisted pipeline is therefore the same computational transformation used during inference.

The test data are not used when the deployment artifact is fitted.

```
[39]: # =====
# 23. Save the complete deployment pipeline
```

```

# =====

import joblib

MODEL_DIR = "model"

os.makedirs(
    MODEL_DIR,
    exist_ok=True
)

final_pipeline = Pipeline([

    (
        "preprocessor",
        preprocessor
    ),

    (
        "model",
        selected_model
    )

])

# -----
# Fit the final pipeline on training data only.
# -----

final_pipeline.fit(

    X_train,

    y_train

)

MODEL_PATH = os.path.join(

    MODEL_DIR,

    "house_price_model_pipeline.joblib"

```

```

)

joblib.dump(
    final_pipeline,
    MODEL_PATH
)

print(
    "Pipeline saved to:"
)

print(
    MODEL_PATH
)

```

Pipeline saved to:  
model\house\_price\_model\_pipeline.joblib

```

[40]: # =====
# Save reproducibility metadata
# =====

metadata = {

    "dataset":
        "London House Price Data",

    "dataset_source":
        "https://www.kaggle.com/datasets/jakewright/house-price-data",

    "target":
        TARGET,

    "selected_model":
        selected_model_name,

    "random_seed":
        RANDOM_SEED,

    "experiment_rows":
        len(df),

```

```

    "train_rows":
        len(X_train),

    "validation_rows":
        len(X_val),

    "test_rows":
        len(X_test),

    "original_feature_dimension":
        int(
            X_engineered.shape[1]
        ),

    "final_feature_dimension":
        int(
            d_final
        ),

    "model_path":
        MODEL_PATH
}

METADATA_PATH = os.path.join(

    MODEL_DIR,

    "metadata.json"

)

with open(

    METADATA_PATH,

    "w",

    encoding="utf-8"

) as file:

    json.dump(

        metadata,

```

```

        file,

        indent=4

    )

print(
    "Metadata saved to:"
)

print(
    METADATA_PATH
)

```

Metadata saved to:  
model\metadata.json

## 1.24 24. Inference Test

### 1.24.1 Question

Can the persisted pipeline be loaded and used on a previously unseen property without retraining?

One observation is taken from the isolated test set.

The persisted pipeline is then reloaded independently and used to generate a prediction.

This verifies that the deployment artifact contains everything required for inference.

```

[41]: # =====
# Reload the persisted pipeline
# =====

loaded_pipeline = joblib.load(
    MODEL_PATH
)

print(
    "Persisted pipeline reloaded successfully."
)

```

Persisted pipeline reloaded successfully.

```

[42]: # =====
# Inference on one unseen test property
# =====

```

```

sample_input = (
    X_test
    .iloc[
        0
    ]
    .copy()
)

sample_actual = (
    y_test
    .iloc[0]
)

sample_prediction = (

    loaded_pipeline
    .predict(
        sample_input
    )[0]

)

sample_error = abs(

    sample_actual
    -
    sample_prediction

)

print(
    "Actual price      :",
    sample_actual
)

print(
    "Predicted price :",
    sample_prediction
)

print(
    "Absolute error   :",
    sample_error
)

```

```
)
```

```
Actual price      : 175000  
Predicted price   : 143491.54694637662  
Absolute error    : 31508.45305362338
```

## 2 Final Summary

The House Price application follows the complete development pipeline:

Raw Data → Cleaning → Representation → Feature Engineering → Preprocessing → Model Training → Evaluation

The original London property records are transformed from a heterogeneous tabular representation into a numerical feature matrix.

Five regression algorithms are compared, and three focused hyperparameter experiments investigate Ridge regularization, Random Forest ensemble size, and Gradient Boosting learning rate.

The final model is selected using validation performance and evaluated on the isolated test set. The selected preprocessing and model are then persisted and verified through an independent inference test.

[ ]: